

Design Patterns

L 01 S04

APDP

Study...

- Aims and benefits of reusable design patterns, General reusable solution, represent an idea and language independent.
- Grouping of design patterns into **creational, structural and behavioural groups.**

"Each pattern describes a problem which occurs over and over again in our environment, and then describes the core of the solution to that problem, in such a way that you can use this solution a million times over, without ever doing it the same way twice"

– Christopher Alexander

Why Design Pattern in the Software Context...

- Design patterns make it easier to reuse successful Designs and Architectures
- Design Pattern → Right Design (Faster)
- A design pattern is a particular approach to solving a recurring problem in software development and is also used to represent good practices. It is not actual code, but it represents a particular way of organizing code to produce the desired solution in a recommended way. This means that we do not have to strictly follow set design patterns for all of our code, but analyze our needs and apply the best-suited design pattern for our needs.

Design Patterns?

- Design patterns arise from a wealth of experience in solving problems in software development and are tried and tested to fit particular scenarios. Another reason to learn about design patterns is that, while a pattern may not work for a particular scenario, it can offer a base off which a solution can be formulated.
- However, as much as design patterns can help us out of various situations, it is important to assess the current situation and explore all the options available before jumping directly into a pattern because there may be a better solution out there.

Advantages of using Design Patterns

- **Reusability:** By using Inheritance, they help in making the code reusable and hence we can use them in multiple projects.
- **Transparent:** It improves the transparency of the code for all the developers who are going to use it in future.
- **Established Solution:** We can blindly believe on the solution provided by **Design Patterns** as they are well-proved and testified at critical stages.
- **Established Communication:** Design patterns make communication between designers more efficient. Software professionals can immediately picture the high-level design in their heads when they refer the name of the pattern used to solve a particular issue when discussing system design.
- **Efficient Development:** Design patterns help in the development of the highly cohesive modules with minimal coupling.

Classification of Design Patterns

		Purpose		
		Creational	Structural	Behavioral
Scope	Class	Factory	Adapter	Interpreter Template Method
	Object	Abstract Factory Builder Prototype Singleton	Bridge Composite Decorator Facade Proxy Flyweight	Chain of Responsibility Command Iterator Mediator Memento Observer State Strategy Visitor

Creational

Factory

Abstract
Factory

Singleton

Prototype

Builder

Structural

Adapter

Bridge

Composite

Decorator

Facade

Flyweight

Proxy

Behavioral

Chain of
Responsibility

Command

Interpreter

Mediator

Memento

Observer

State

Strategy

Template

Visitor

Creational

- Creational Design Patterns deal with creation of classes or objects. They serve to abstract away the specifics of classes so that we'd be less dependent on their exact implementation, or so that we wouldn't have to deal with complex construction whenever we need them, or so we'd ensure some special instantiation properties.
- They're very useful for lowering levels of dependency and controlling how the user interacts with our classes.

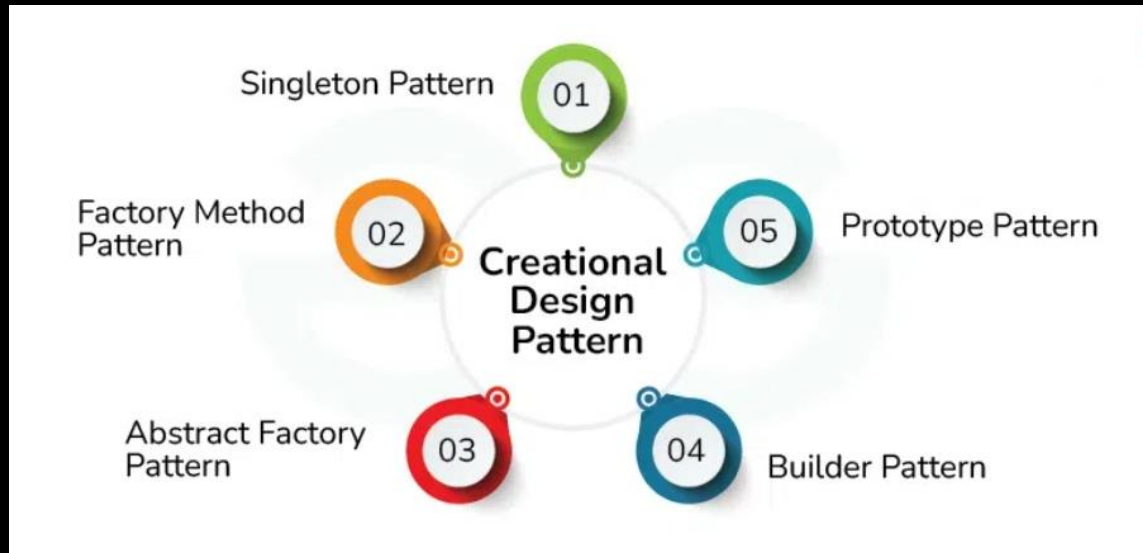
Structural

- Structural Design Patterns deal with assembling objects and classes into larger structures, while keeping those structures flexible and efficient.
- They tend to be really useful for improving readability and maintainability of the code, ensure functionalities are properly separated, encapsulated, and that there are effective minimal interfaces between interdependent things.

Behavioral

- Behavioral Design Patterns deal with algorithms in general, and assignment of responsibility between interacting objects.
- For example, they're good practices in cases where you may be tempted to implement a naive solution, like busy waiting, or load your classes with unnecessary code for one specific purpose that isn't the core of their functionality.

Classification of Creational Design Patterns



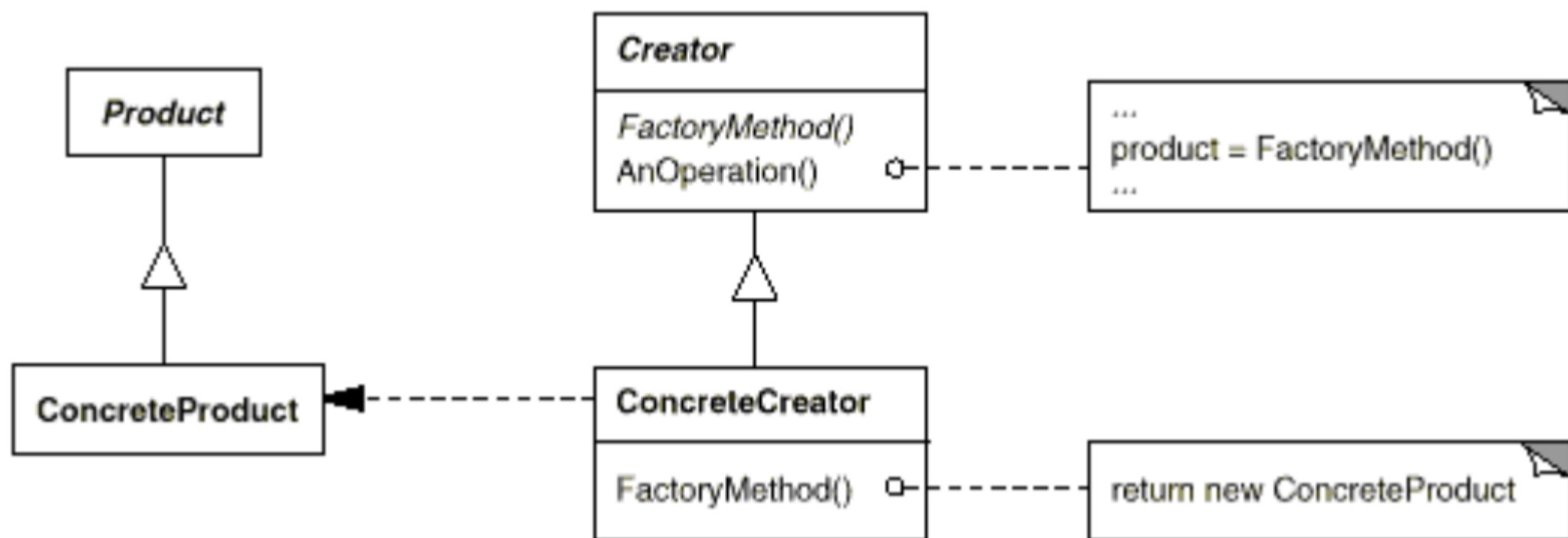
- Factory Method
- Abstract Factory Method
- Builder Method
- Singleton Method
- Prototype Method

1. Factory Method

- The Factory Method is used in object-oriented programming as a means to provide factory interfaces for creating objects. These interfaces define the generic structure, but don't initialize objects. The initialization is left to more specific subclasses.
- The parent class/interface houses all the standard and generic behavior that can be shared across subclasses of different types. The subclass is in turn responsible for the definition and instantiation of the object based on the superclass.

Scope of Factory Design Pattern

- The main motivation behind the Factory Method Design Pattern is to enhance loose coupling in code through the creation of an abstract class that will be used to create different types of objects that share some common attributes and functionality.
- This results in increased flexibility and reuse of code because the shared functionality will not be rewritten having been inherited from the same class. This design pattern is also known as a Virtual Constructor.
- The Factory Method design pattern is commonly used in libraries by allowing clients to choose what subclass or type of object to create through an abstract class.
- A Factory Method will receive information about a required object, instantiate it and return the object of the specified type. This gives our application or library a single point of interaction with other programs or pieces of code, thereby encapsulating our object creation functionality.



Components

1. Creator

This is an abstract class or an interface that declares the factory method. The creator typically contains a method that serves as a factory for creating objects. It may also contain other methods that work with the created objects.

2. Concrete Creator

Concrete Creator classes are subclasses of the Creator that implement the factory method to create specific types of objects. Each Concrete Creator is responsible for creating a particular product.

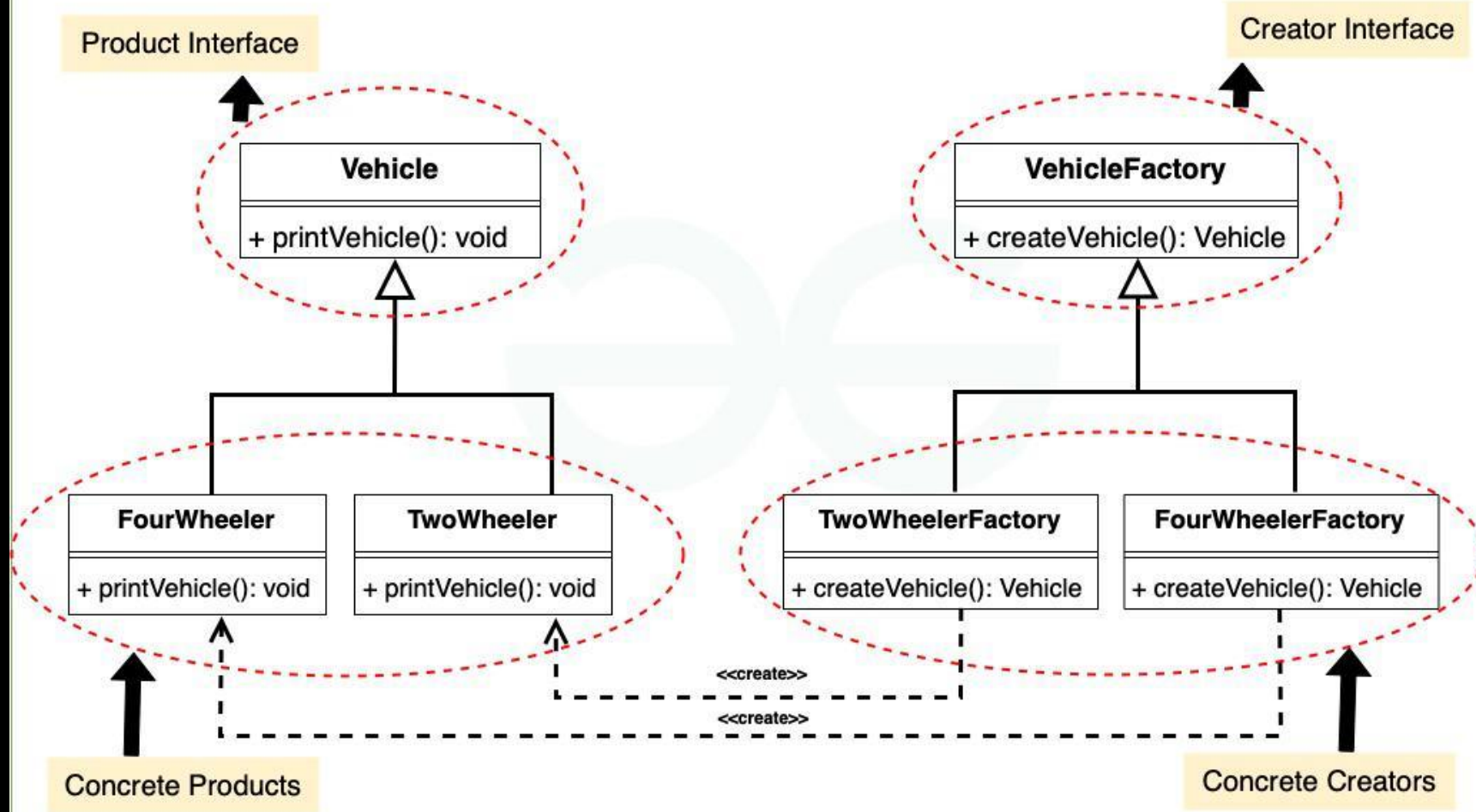
3. Product

This is the interface or abstract class for the objects that the factory method creates. The Product defines the common interface for all objects that the factory method can create.

4. Concrete Product

Concrete Product classes are the actual objects that the factory method creates. Each Concrete Product class implements the Product interface or extends the Product abstract class.

Factory Method Design Pattern Class Diagram



Advantages of using Factory method

- We can easily add new types of products without disturbing the existing client code.
- Generally, tight coupling is being avoided between the products and the creator classes and objects.

Disadvantages of using Factory method

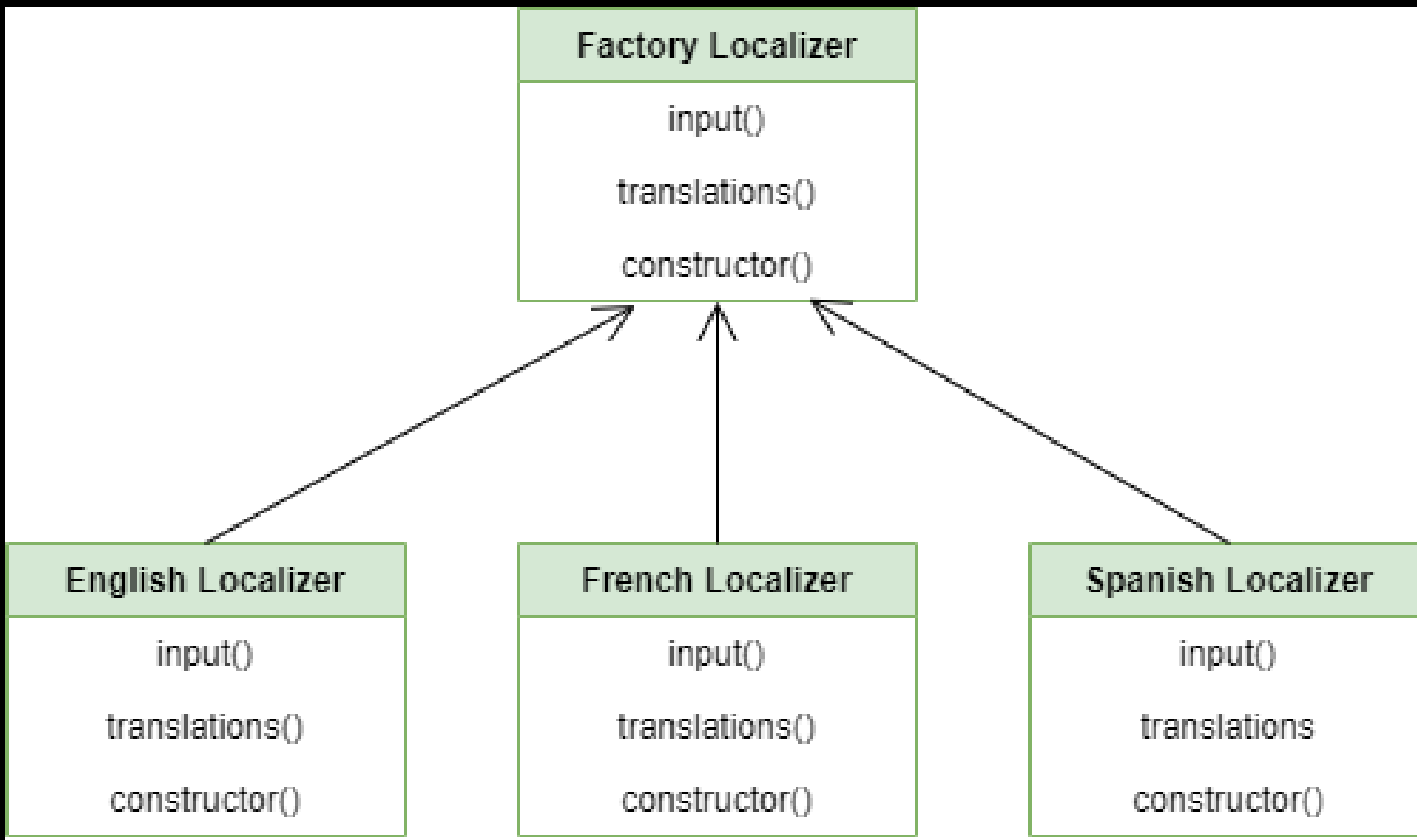
- To create a particular concrete product object, the client might have to sub-class the creator class.
- You end up with a huge number of small files

Example

- Consider a software application that do Best-Selling Products Analysis, different types of products (e.g. bakery, beverage, pharmacy, meat, sea food) will have distinct properties and behaviors related to how their best-selling status is determined. For each product category, the criteria to determine the best-selling products may differ, so each product type will have its own specific logic for analyzing sales.
- **Implement a code for the above application using Factory Design Pattern.**

Exercise

- Create a product class
- Create concrete products (subclasses of Sales)
- Create a factory class
- Create concrete factories (subclasses of SalesFactory)
- Client code
- Client chooses to process an online sale
- Client chooses to process an in-store sale



2. Abstract Factory Method

- The Abstract Factory Design Pattern is a creational pattern that provides a framework to create interfaces of closely related objects without specifying their classes - hence the term "abstract".
- An abstract class is partially implemented and defines the requirements that its child classes should have and some generic child behavior as well as what functions they should have. Concrete classes extend abstract classes and provide the unimplemented functionality, while inheriting the common functionalities.

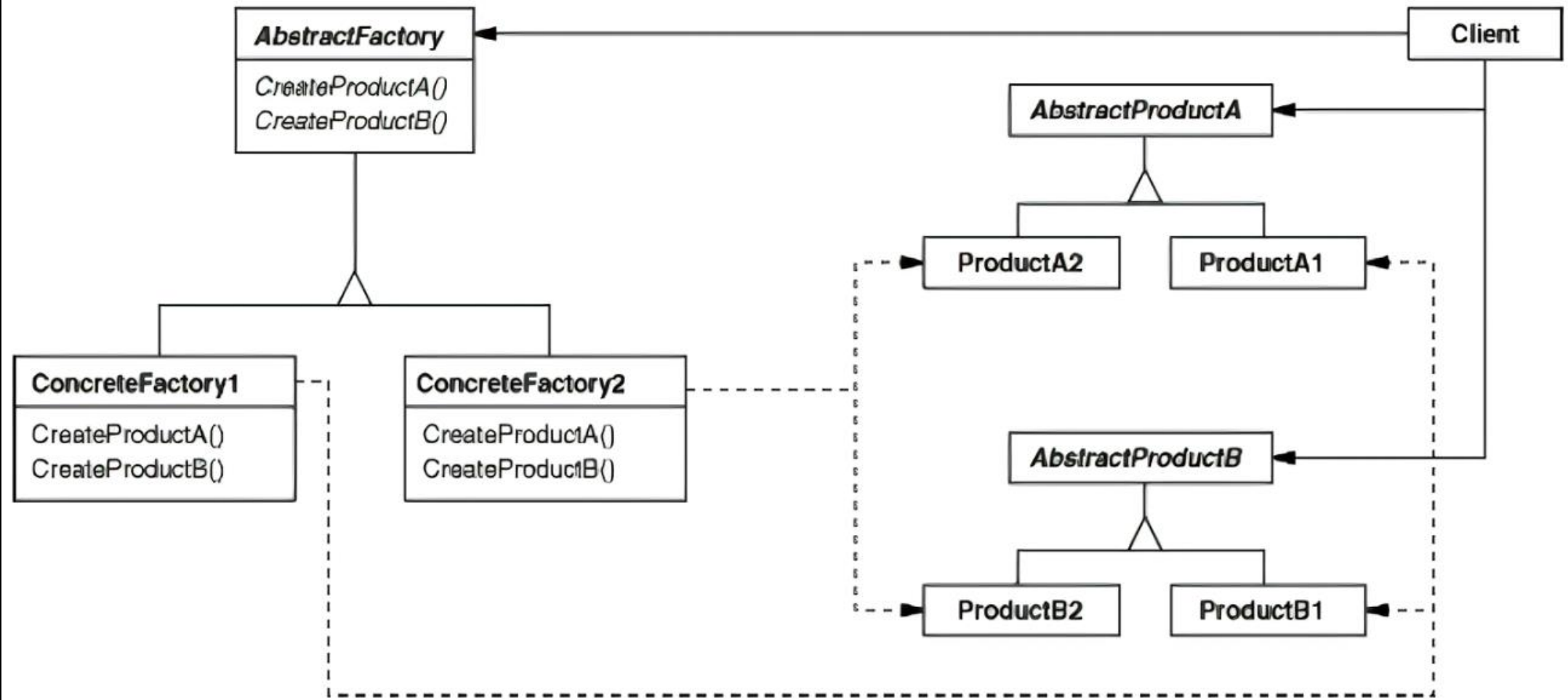
- Abstract Factory Method allows you to produce the families of related objects without specifying their concrete classes. Using the abstract factory method, we have the easiest ways to produce a similar type of many objects.
- It provides a way to encapsulate a group of individual factories. Basically, here we try to abstract the creation of the objects depending on the logic, business, platform choice, etc.

Example

- All **Animals** have an **eat()** function, commonly implemented between all animals, but each have unique **speak()** functions so the **abstract Animal class** leaves that up to the children.

Components

1. Abstract Product
2. Concrete Product
3. Abstract Factory
4. Concrete Factory
5. Client



Example: - Solution by using Abstract Factory Method

Assume that you are designing a family of two products (a browser and a messenger),

Abstract Products: Two abstract classes are created, one for the browser and another for the messenger. These classes contain abstract methods that are mandatory for the construction of the products. These abstract classes are referred to as interfaces.

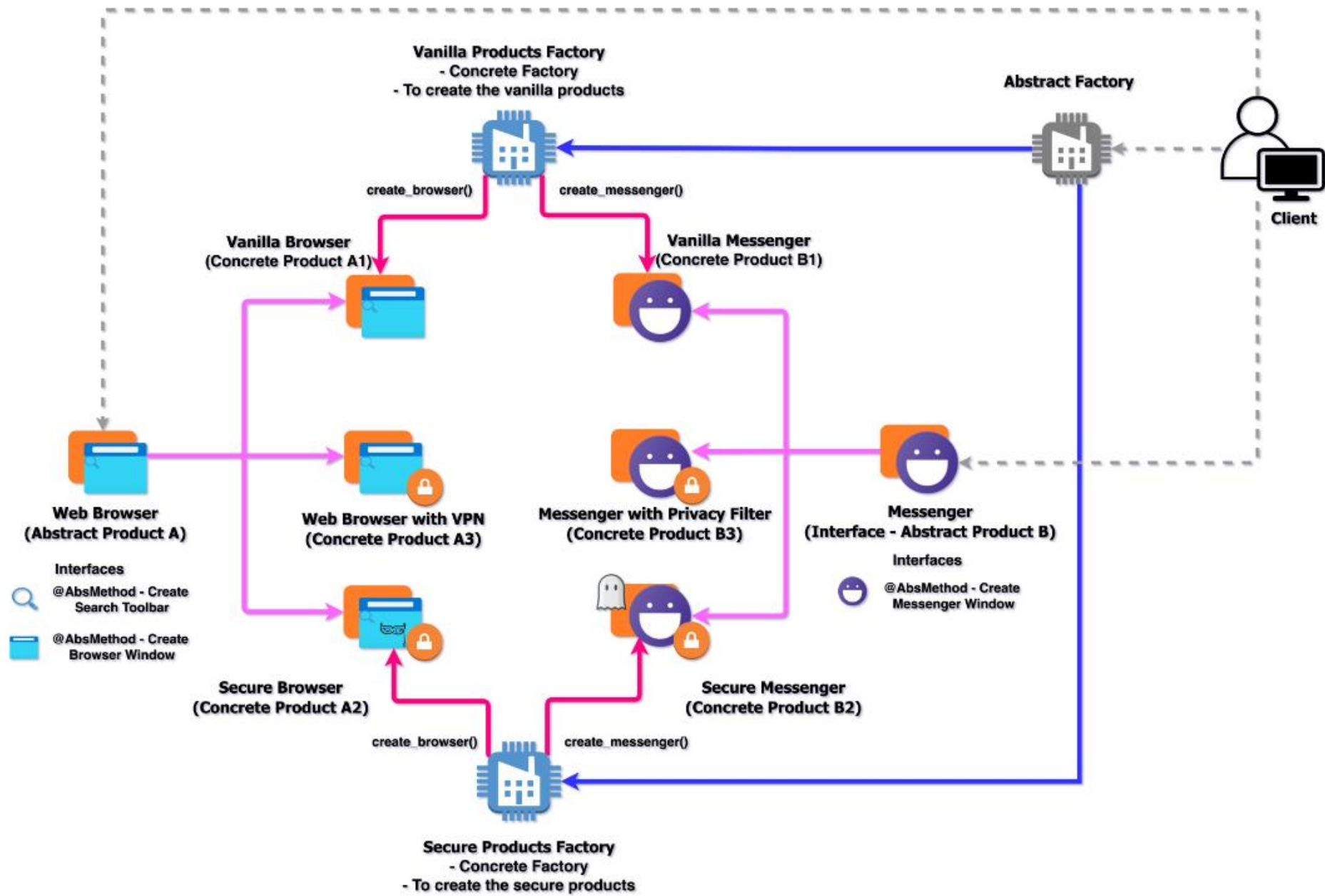
In the example shown above, the Web Browser and Messenger are the abstract products.

Concrete Products: Concrete products inherit the abstract methods from the abstract classes i.e. abstract products. Using the interfaces, different families of products can be created.

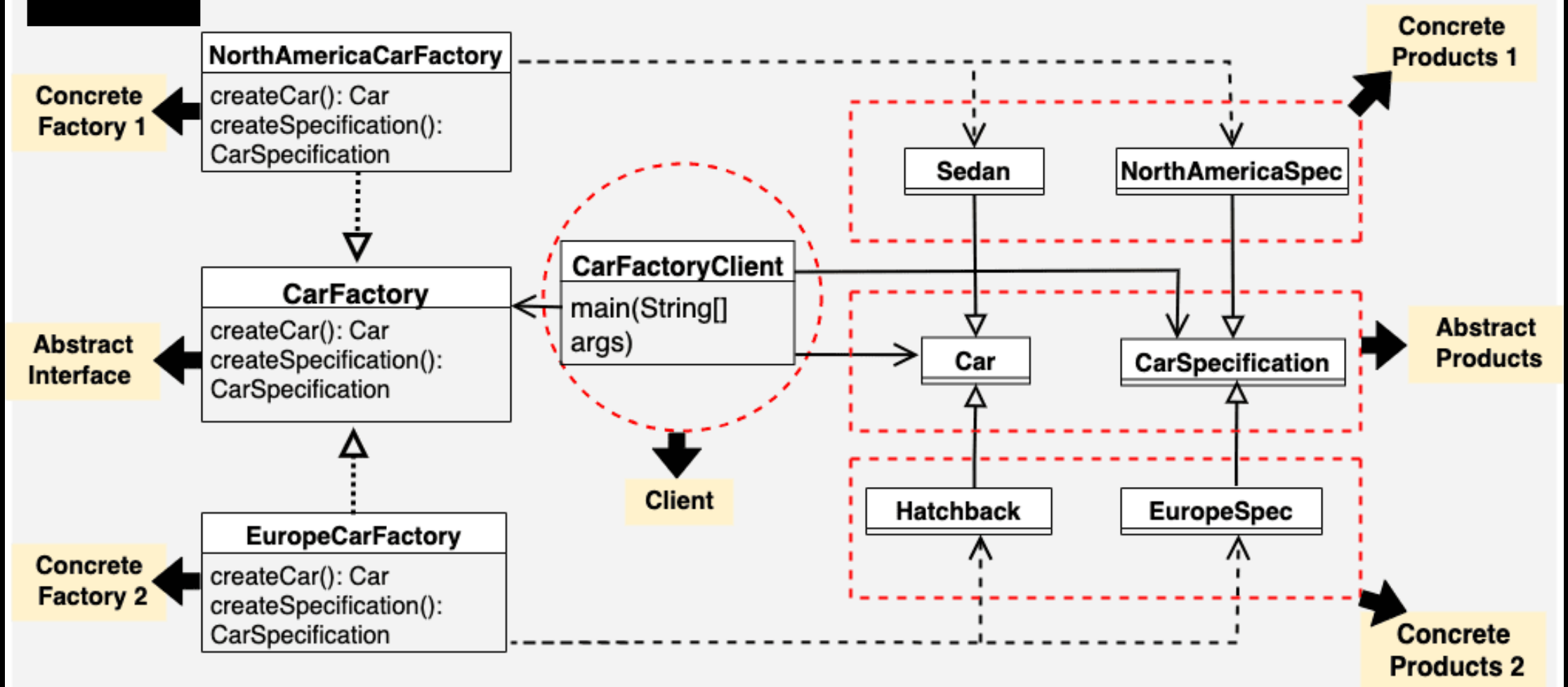
For example, in the diagram above, three different kinds of web browsers are created for three different sets of users. If there's one thing that all these concrete products have in common, that would be the abstract methods that were defined in the abstract class.

Solution by using Abstract Factory Method

- **Concrete Factories:** Concrete Factories create Concrete Products as directed by the Abstract Factories. The concrete factories are only capable of creating those products that are specified in them - a BrowserFactory creates browsers, while a MessengerFactory creates messengers. Alternatively, you can focus on some common features, and say - create a BasicFactory and SecureFactory which create basic or secure web browsers and messenger instances.
- In the diagram mentioned above, the Vanilla Products Factory is capable of creating both vanilla concrete products (browser and messenger), while the Secure Products Factory makes safe versions.
- **Abstract Factories:** The Abstract factories possess interfaces to create the abstract products i.e. they contain several methods that return abstract products.

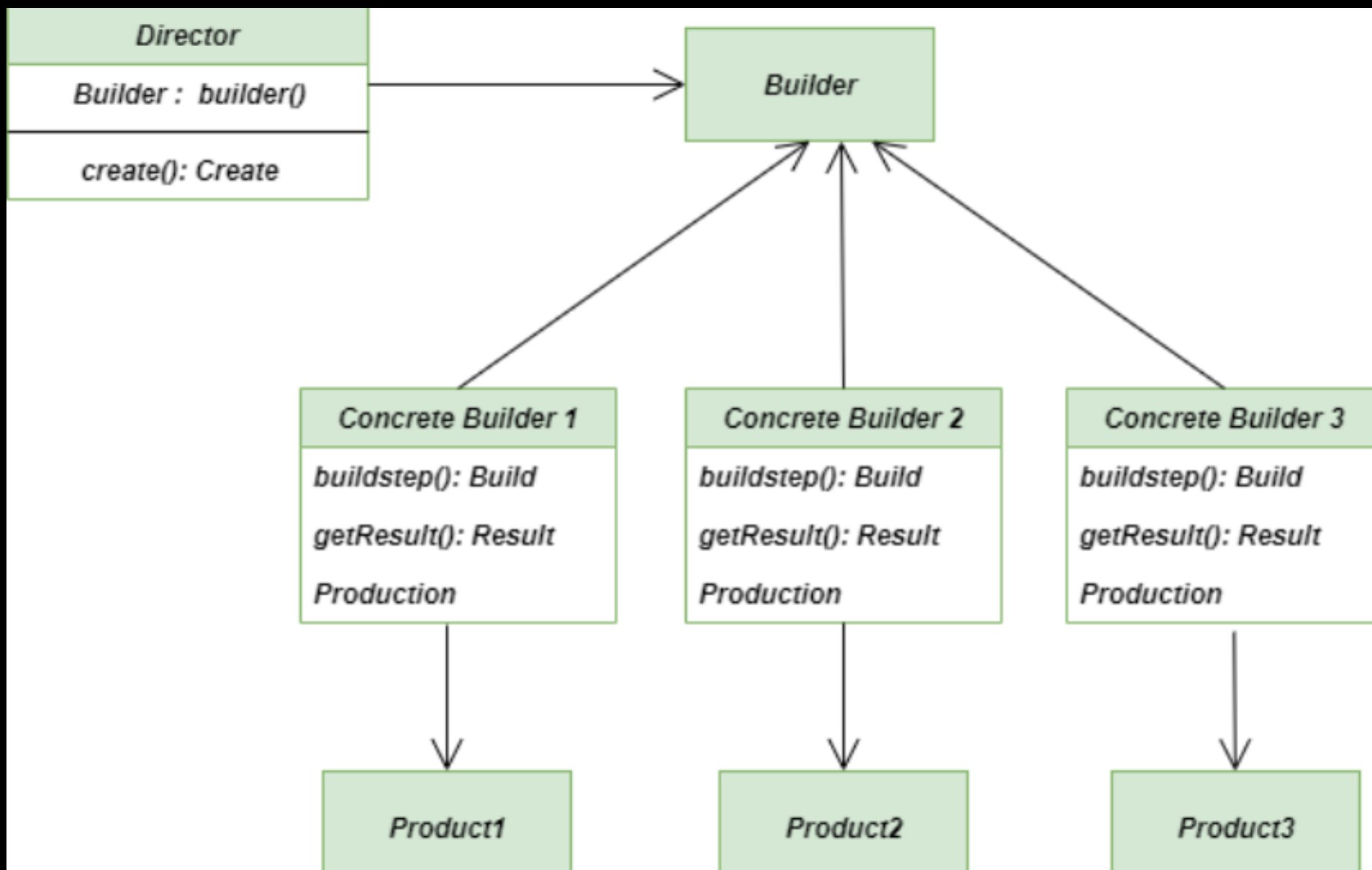


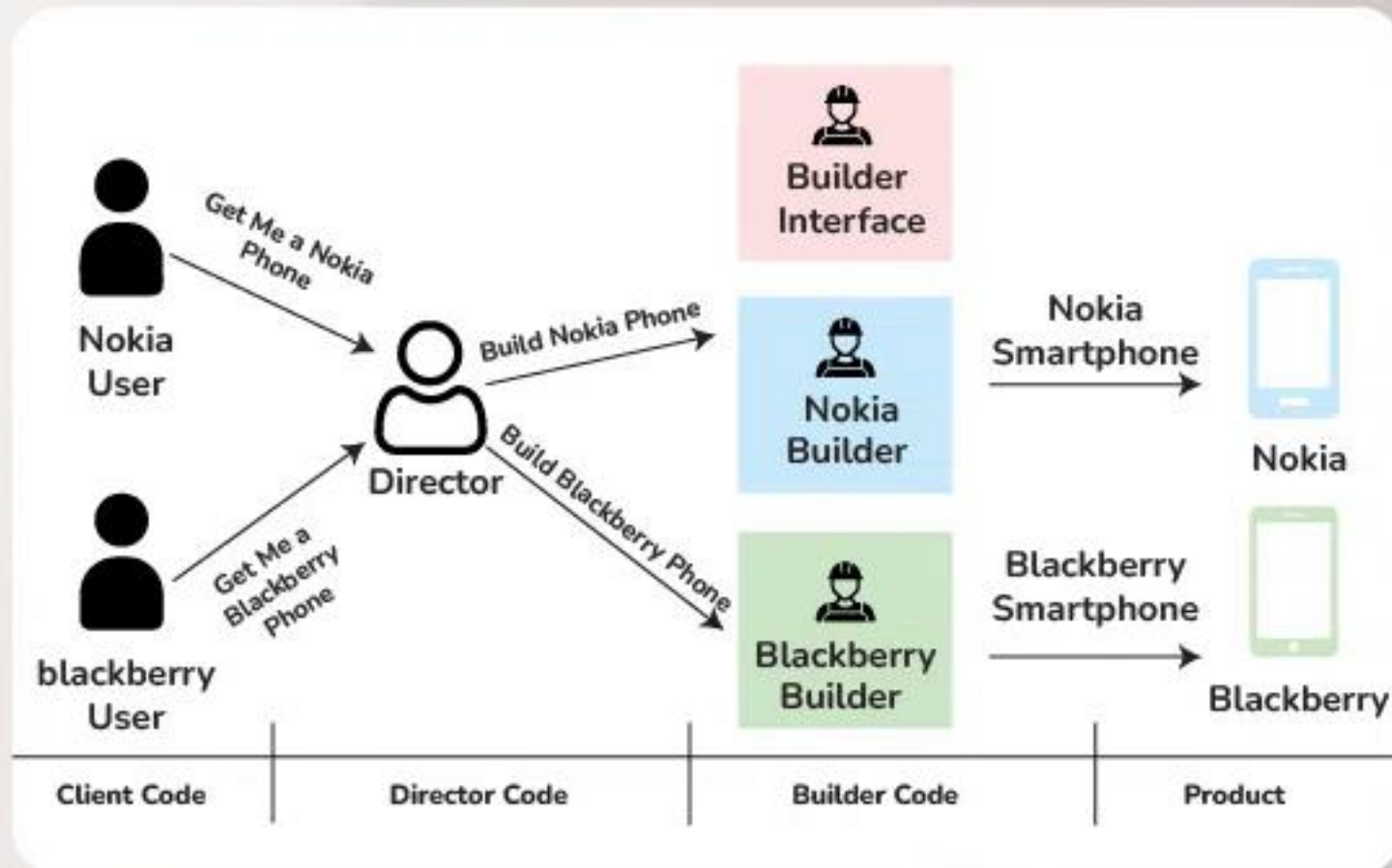
Class Diagram of Abstract Factory Pattern



3. Builder Method

- Builder is a creational design pattern that lets you **construct complex objects step by step**. The pattern allows you to produce different types and representations of an object using the same construction code.
- Sometimes, the objects we create can be complex, made up of several sub-objects or require an elaborate construction process. The exercise of creating complex types can be simplified by using the builder pattern. A **composite or an aggregate object** is what a builder generally builds
- A builder pattern encapsulates or hides the process of building a complex object and **separates the representation of the object and its construction**. The separation allows us to construct different representations using the same construction process





Components of the Builder Design Pattern

1. Product

- The Product is the complex object that the Builder pattern is responsible for constructing.
- It may consist of multiple components or parts, and its structure can vary based on the implementation.
- The Product is typically a class with attributes representing the different parts that the Builder constructs.

2. Builder

- The Builder is an interface or an abstract class that declares the construction steps for building a complex object.
- It typically includes methods for constructing individual parts of the product.
- By defining an interface, the Builder allows for the creation of different concrete builders that can produce variations of the product.

3. ConcreteBuilder

- ConcreteBuilder classes implement the Builder interface, providing specific implementations for building each part of the product.
- Each ConcreteBuilder is tailored to create a specific variation of the product.
- It keeps track of the product being constructed and provides methods for setting or constructing each part.

4. Director

- The Director is responsible for managing the construction process of the complex object.
- It collaborates with a Builder, but it doesn't know the specific details about how each part of the object is constructed.
- It provides a high-level interface for constructing the product and managing the steps needed to create the complex object.

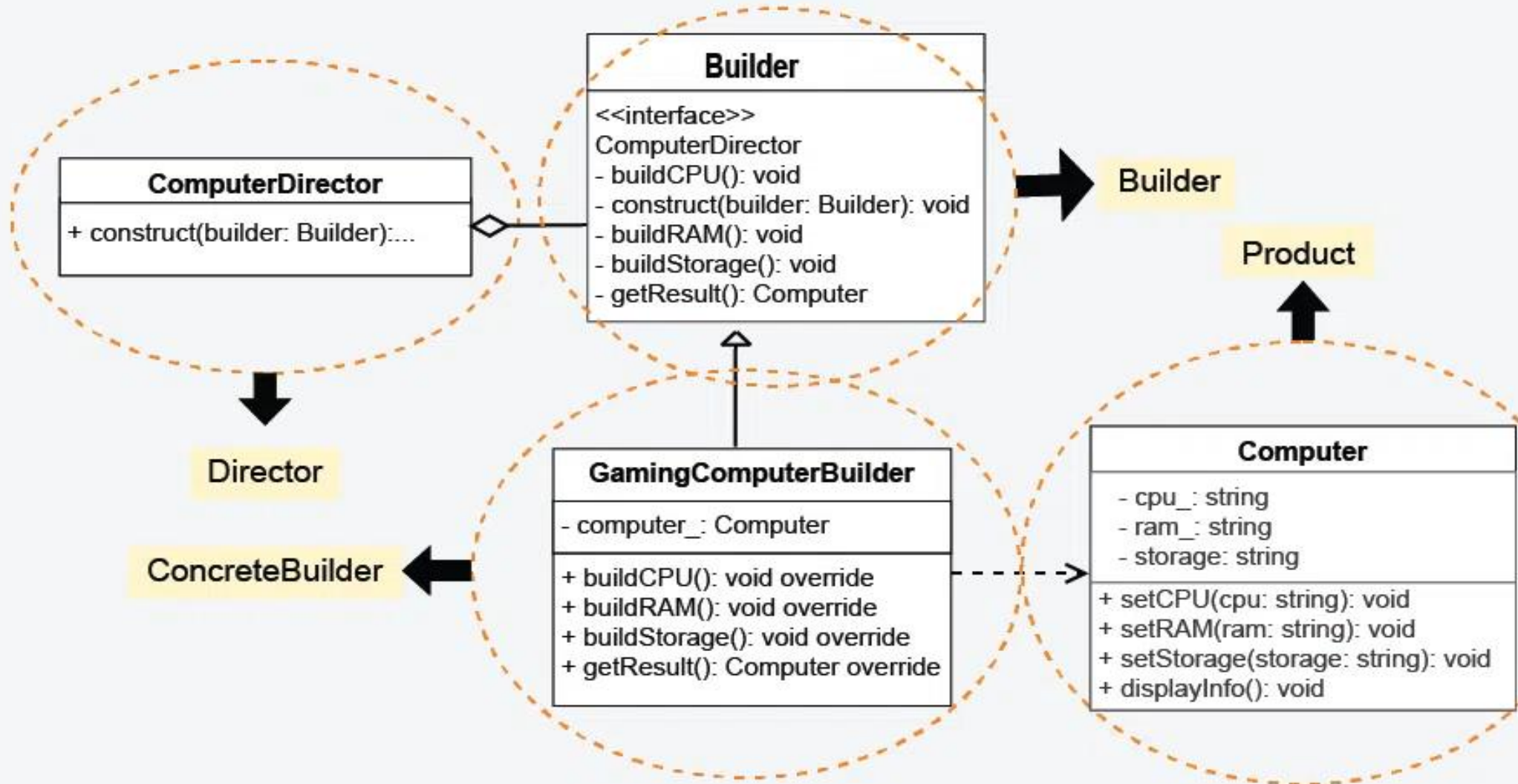
5. Client

- The Client is the code that initiates the construction of the complex object.
- It creates a Builder object and passes it to the Director to initiate the construction process.
- The Client may retrieve the final product from the Builder after construction is complete.

Problem Scenario 1

- You are tasked with implementing a system for building custom computers. Each computer can have different configurations based on user preferences. The goal is to provide flexibility in creating computers with varying CPUs, RAM, and storage options.
- Implement the Builder design pattern to achieve this, allowing the construction of computers through a step-by-step process. Use the provided components – Product (Computer), Builder interface, ConcreteBuilder (GamingComputerBuilder), Director, and Client

UML Class Diagram for Builder Design Pattern



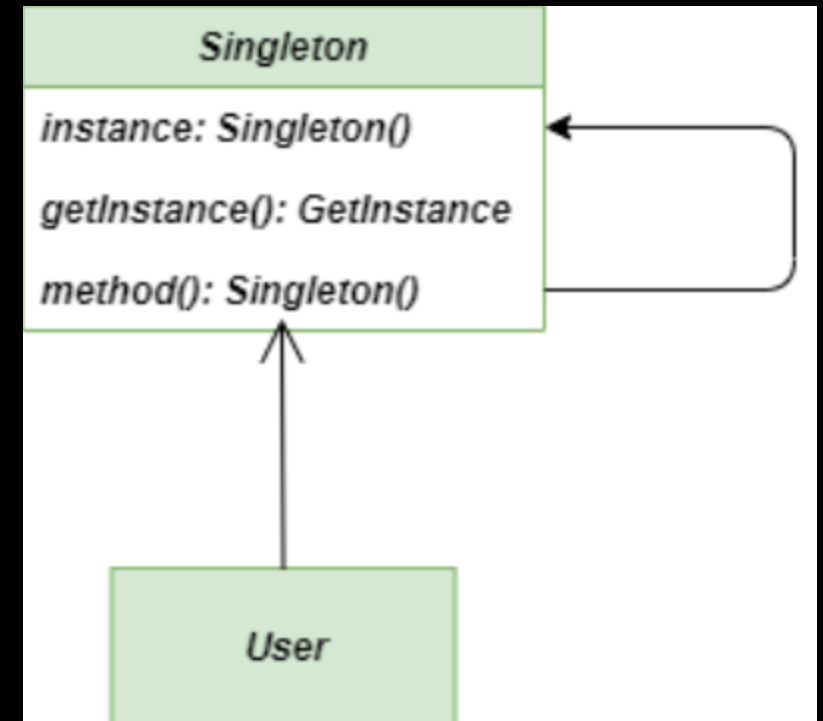
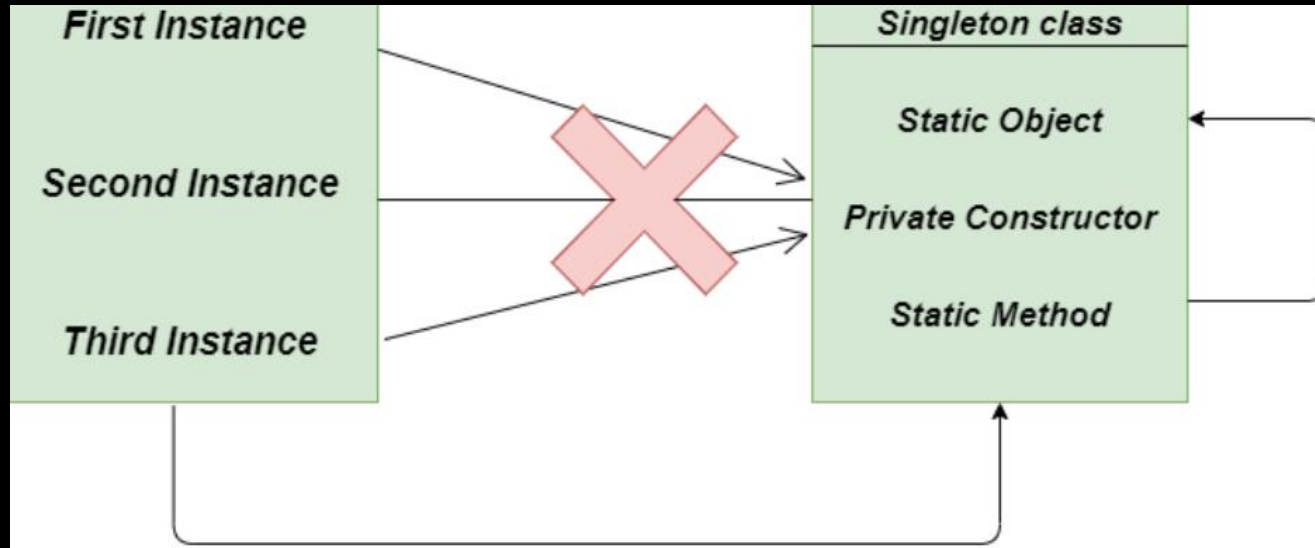
Problem Scenario 2

- Say you need to represent a robot with your object structure. The robot can be humanoid with four limbs and upward standing, or it can be animal-like with a tail, wings, etc. It can use wheels to move, or it can use helicopter blades. It can use cameras, an infrared detection module, and so on. You get the picture.

04. Singleton Method

- Singleton Method is one of the simplest design patterns available to us. It is a way to provide one and only one object of a particular type.
- It involves only one class to create methods and specify the objects.
- Singleton Design Pattern can be understood by a very simple example of Database connectivity. When each object creates a unique Database Connection to the Database, it will highly affect the cost and expenses of the project. So, it is always better to make a single connection rather than making extra irrelevant connections which can be easily done by **Singleton Design Pattern**.

Design



Creating a singleton in Python

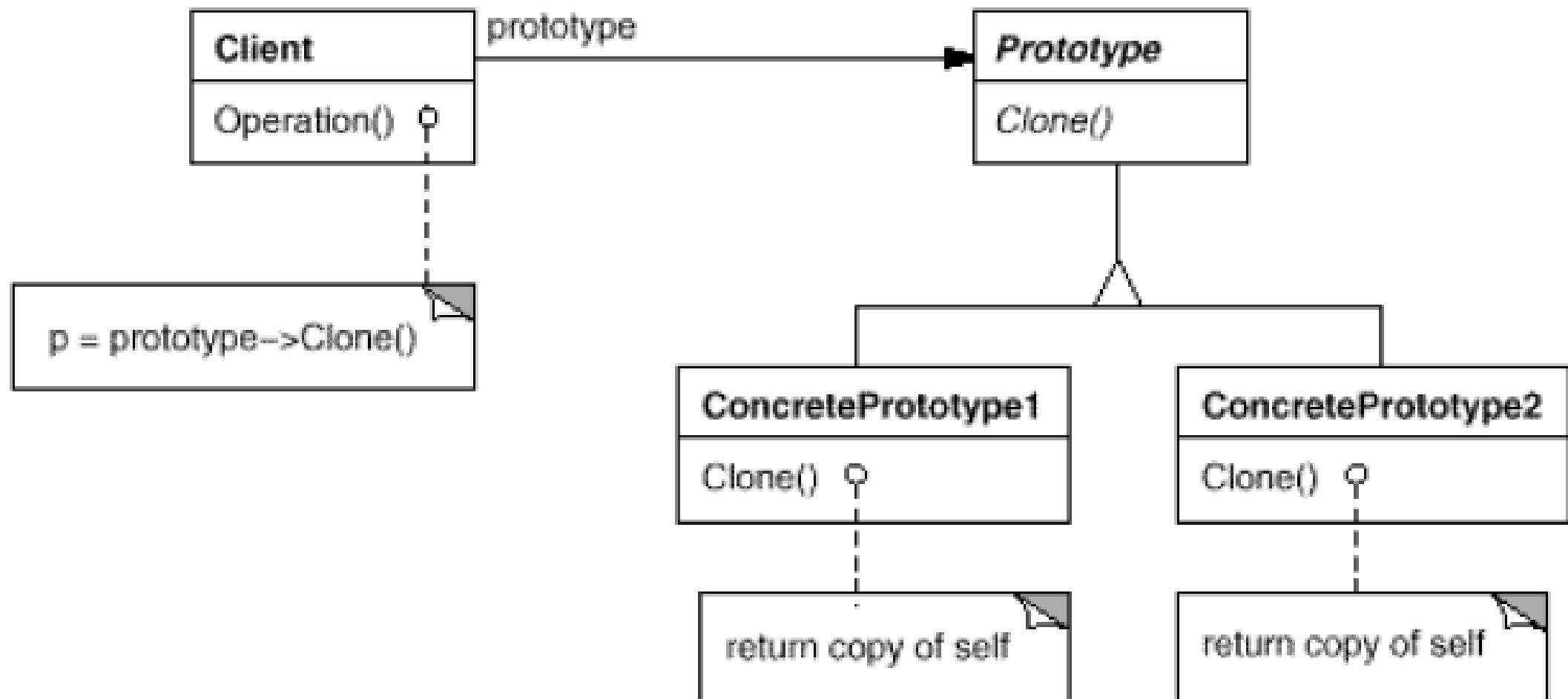
The Singleton pattern solves two problems at the same time, violating the Single Responsibility Principle

1. **Ensure that a class has just a single instance - Having only one instance** is usually a mechanism for controlling access to some shared resource. For example, two threads may work with the same file, so instead of both opening it separately, a Singleton can provide a unique access point to both of them.
2. **Provide a global access point to that instance - Global accessibility** is important because after your class has been instantiated once, you'd need to pass that single instance around in order to work with it. It can't be instantiated again. That's why it's easier to make sure that whenever you try to instantiate the class again, you just get the same instance you've already had.

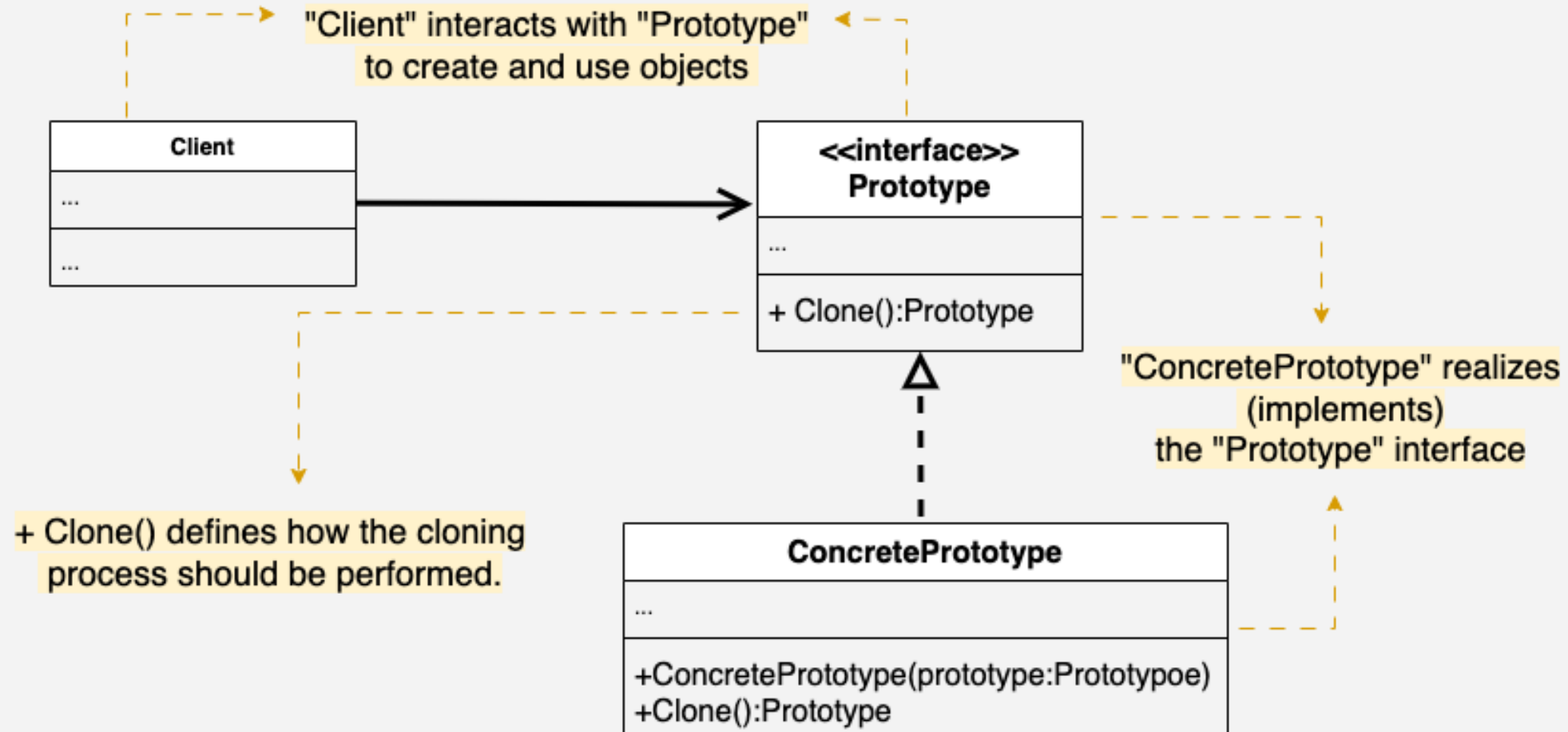
In the classic implementation of the Singleton Design pattern, we simply use the static method for creating the **getInstance** method which has the ability to return the shared resource. We also make use of the so-called **Virtual private Constructor** to raise the exception against it although it is not much required.

5. Prototype Method

- Prototype Method aims to reduce the number of classes used for an application. It allows you to copy existing objects independent of the concrete implementation of their classes. Generally, here the object is created by copying a prototypical instance during run-time.
- It is highly recommended to **use Prototype Method when the object creation is an expensive task** in terms of time and usage of resources and already there exists a similar object. This method provides a way to copy the original object and then modify it according to our needs.
- The Prototype design pattern addresses the problem of copying objects by delegating it to the objects themselves. All objects that are copyable must implement a **method called clone** and use it to return exact copies of themselves.



Class Diagram of Prototype Design Pattern



Components of Prototype Design Pattern

1. Prototype Interface or Abstract Class

- The Prototype Interface or Abstract Class declares the method(s) for cloning an object. It defines the common interface that concrete prototypes must implement, ensuring that all prototypes can be cloned in a consistent manner.
- The main role is to provide a blueprint for creating new objects by specifying the cloning contract.
- It declares the clone method, which concrete prototypes implement to produce copies of themselves.

2. Concrete Prototype

- The Concrete Prototype is a class that implements the prototype interface or extends the abstract class. It's the class representing a specific type of object that you want to clone.
- It defines the details of how the cloning process should be carried out for instances of that class.
- Implements the clone method declared in the prototype interface, providing the cloning logic specific to the class.

3. Client

- The Client is the code or module that requests the creation of new objects by interacting with the prototype. It initiates the cloning process without being aware of the concrete classes involved.

4. Clone Method

- The Clone Method is declared in the prototype interface or abstract class. It specifies how an object should be copied or cloned. Concrete prototypes implement this method to define their unique cloning behavior. It Describes how the object's internal state should be duplicated to create a new, independent instance.

Problem Scenario

- Prototype is a creational design pattern that lets you copy existing objects without making your code dependent on their classes.
- Say you have an object, and you want to create an exact copy of it. How would you do it? First, you have to create a new object of the same class. Then you have to go through all the fields of the original object and copy their values over to the new object.
- Nice! But there's a catch. Not all objects can be copied that way because some of the object's fields may be private and not visible from outside of the object itself.

- There's one more problem with the direct approach. Since you have to know the object's class to create a duplicate, your code becomes dependent on that class. If the extra dependency doesn't scare you, there's another catch. Sometimes you only know the interface that the object follows, but not its concrete class, when, for example, a parameter in a method accepts any objects that follow some interface.

Solution

- The Prototype pattern delegates the cloning process to the actual objects that are being cloned. The pattern declares a common interface for all objects that support cloning. This interface lets you clone an object without coupling your code to the class of that object. Usually, such an interface contains just a single clone method.
- The implementation of the clone method is very similar in all classes. The method creates an object of the current class and carries over all of the field values of the old object into the new one. You can even copy private fields because most programming languages let objects access private fields of other objects that belong to the same class.
- An object that supports cloning is called a prototype. When your objects have dozens of fields and hundreds of possible configurations, cloning them might serve as an alternative to subclassing.
- Here's how it works: you create a set of objects, configured in various ways. When you need an object like the one you've configured, you just clone a prototype instead of constructing a new object from scratch.

02. Structural Design Patterns

- Structural design patterns are about organizing different classes and objects to form larger structures and provide new functionality while keeping these structures flexible and efficient.
- Mostly they use Inheritance to compose all the interfaces.
- It also identifies the relationships which led to the simplification of the structure.

Structural Design Patterns

1. Adapter Method
2. Bridge Method
3. Composite Method
4. Decorator Method
5. Facade Method
6. Proxy Method
7. FlyWeight Method

1. Adapter method

Adapter method is a Structural Design Pattern which helps us in making the incompatible objects adaptable to each other.

The Adapter method is one of the easiest methods to understand because we have a lot of real-life examples that show the analogy with it.

The Adapter design pattern is used to allow incompatible interfaces to work together. It acts as a bridge between two incompatible interfaces by converting the interface of a class into another interface expected by the client. This method provides a different interface for a class.

We can more easily understand the concept by thinking about the Cable Adapter that allows us to charge a phone somewhere that has outlets in different shapes. Using this idea, we can integrate the classes that couldn't be integrated due to interface incompatibility.

Components of Adapter Design Pattern

1. Target Interface

Defines the interface expected by the client. It represents the set of operations that the client code can use. It's the common interface that the client code interacts with.

2. Adaptee

The existing class or system with an incompatible interface that needs to be integrated into the new system. It's the class or system that the client code cannot directly use due to interface mismatches.

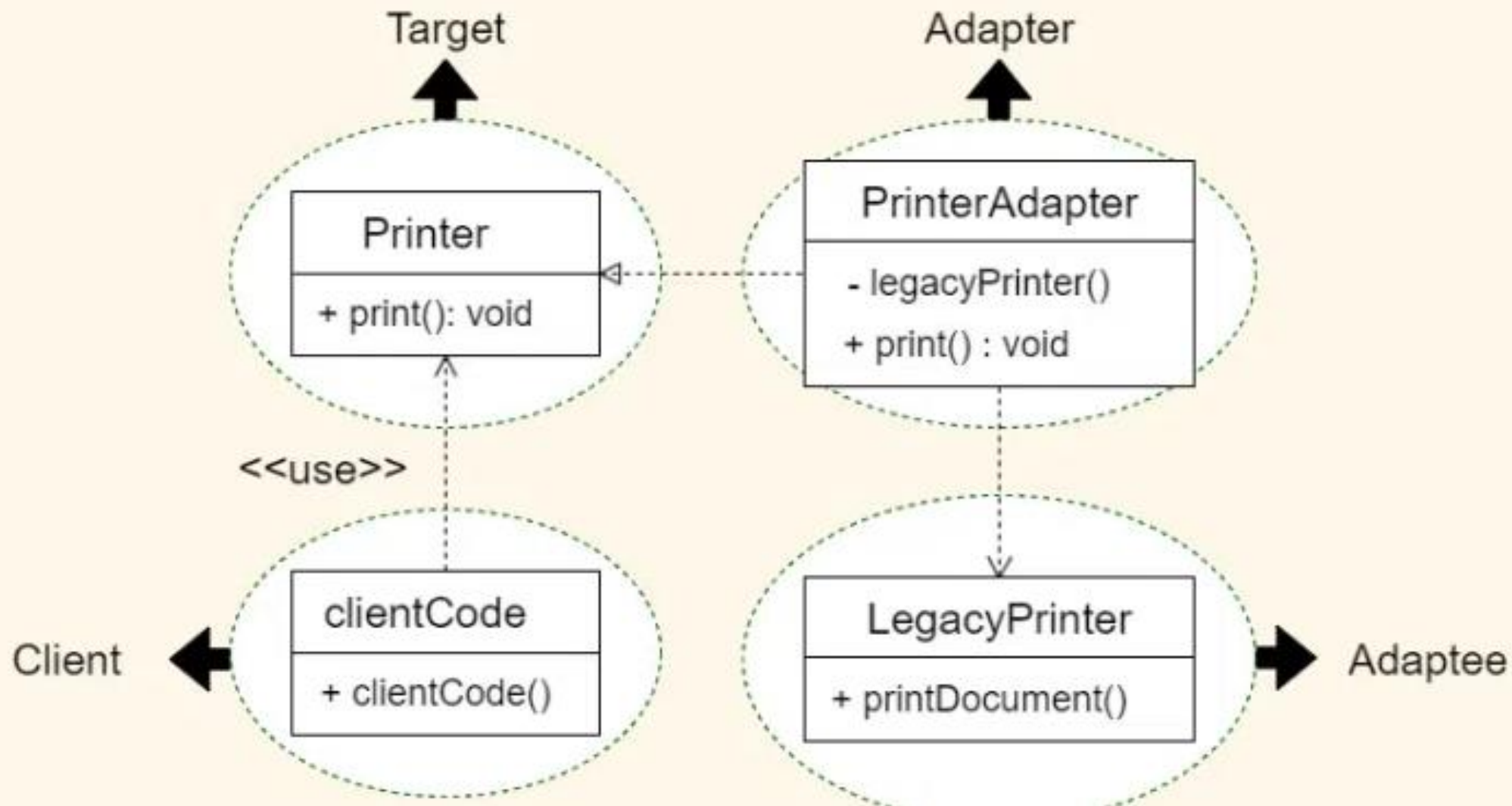
3. Adapter

A class that implements the target interface and internally uses an instance of the adaptee to make it compatible with the target interface. It acts as a bridge, adapting the interface of the adaptee to match the target interface.

4. Client

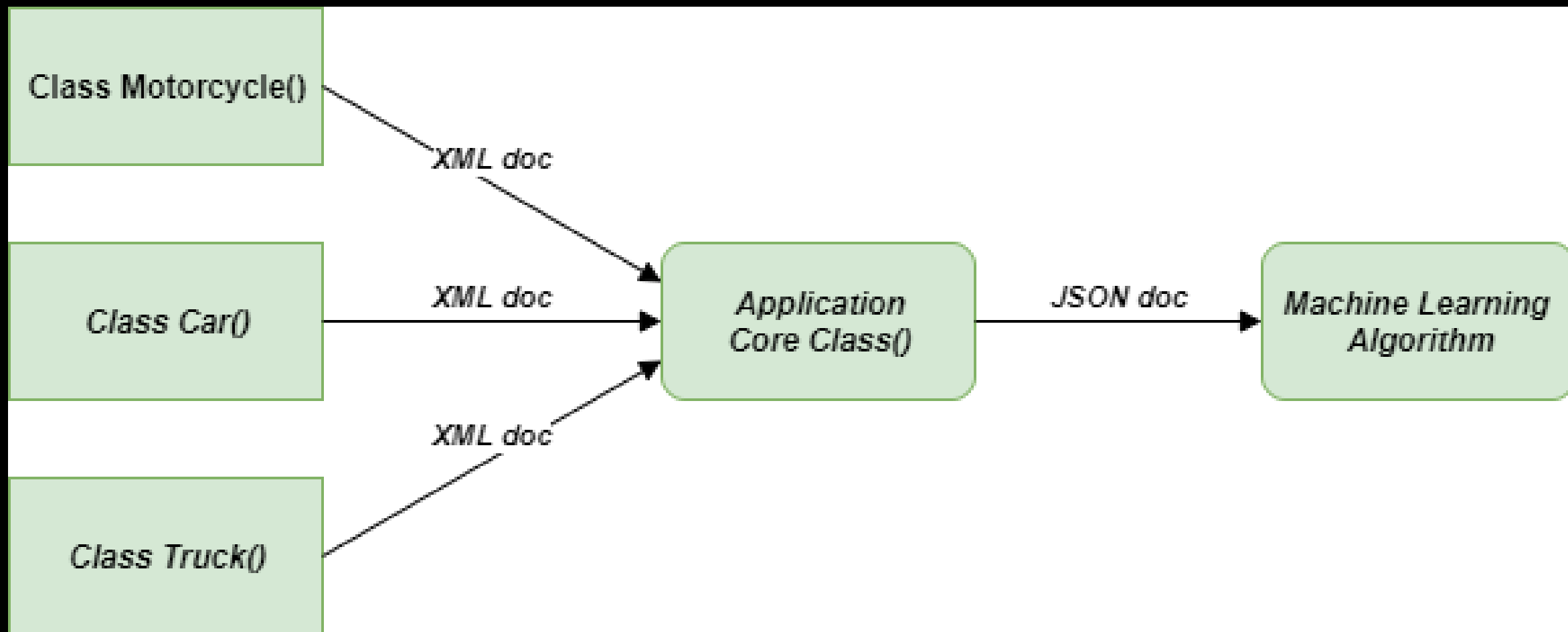
The code that uses the target interface to interact with objects. It remains unaware of the specific implementation details of the adaptee and the adapter. It's the code that benefits from the integration of the adaptee into the system through the adapter.

Class Diagram of Adapter Design Pattern



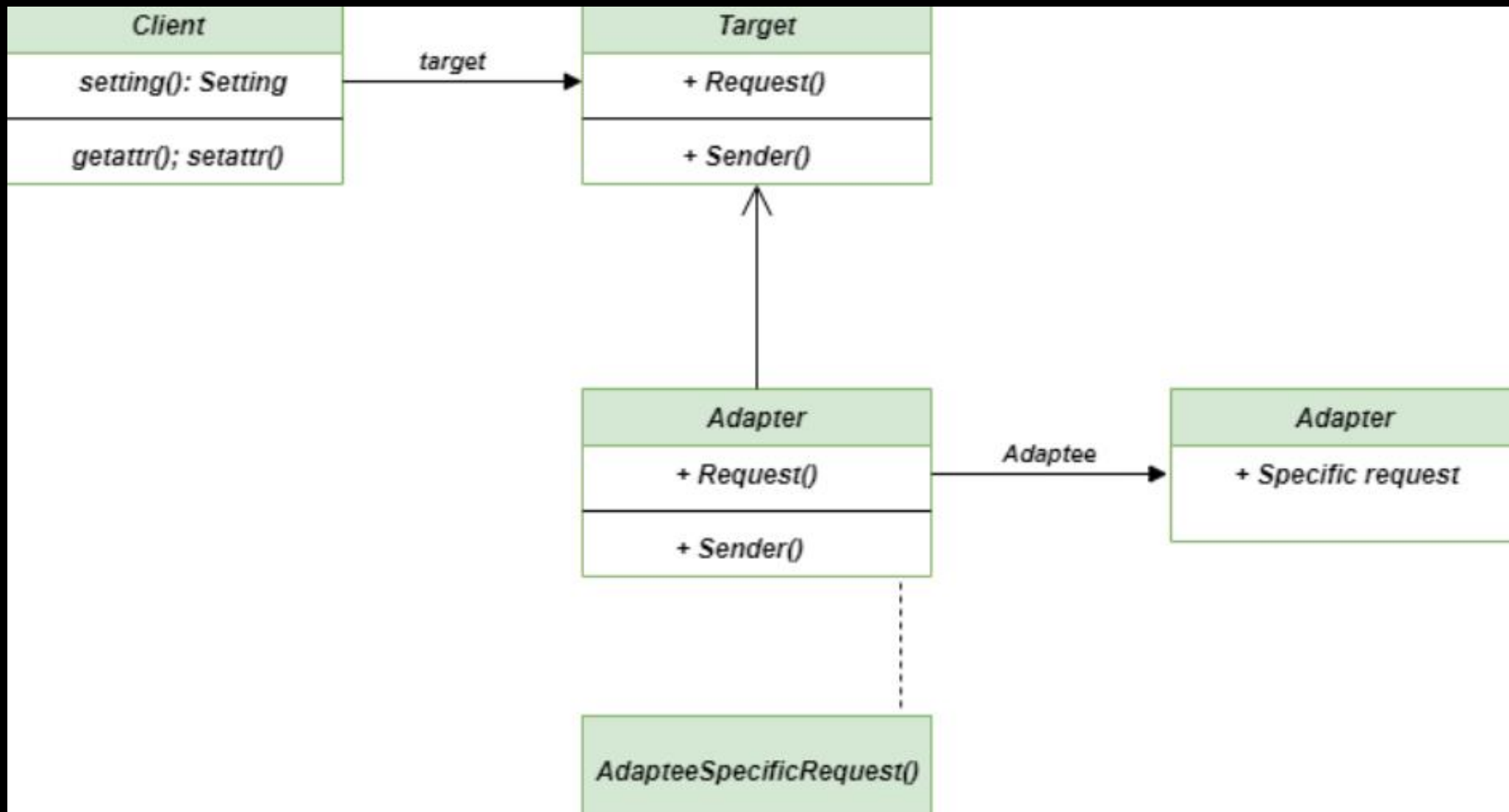
Problem without using the Adapter Method

- Imagine you are creating an application that shows the data about all different types of vehicles present. It takes the data from APIs of different vehicle organizations in XML format and then displays the information. But suppose at some time you want to upgrade your application with a Machine Learning algorithms that work beautifully on the data and fetch the important data only. But there is a problem, it Imagine you are creating an application that shows the data about all different types of vehicles present.
- It takes the data from APIs of different vehicle organizations in XML format and then displays the information. But suppose at some time you want to upgrade your application with a Machine Learning algorithms that work beautifully on the data and fetch the important data only. But there is a problem, it takes data in JSON format only. It will be a really poor approach to make changes in Machine Learning Algorithm so that it will take data in XML formattakes data in JSON format only. It will be a really poor approach to make changes in Machine Learning Algorithm so that it will take data in XML format



Solution using Adapter Method

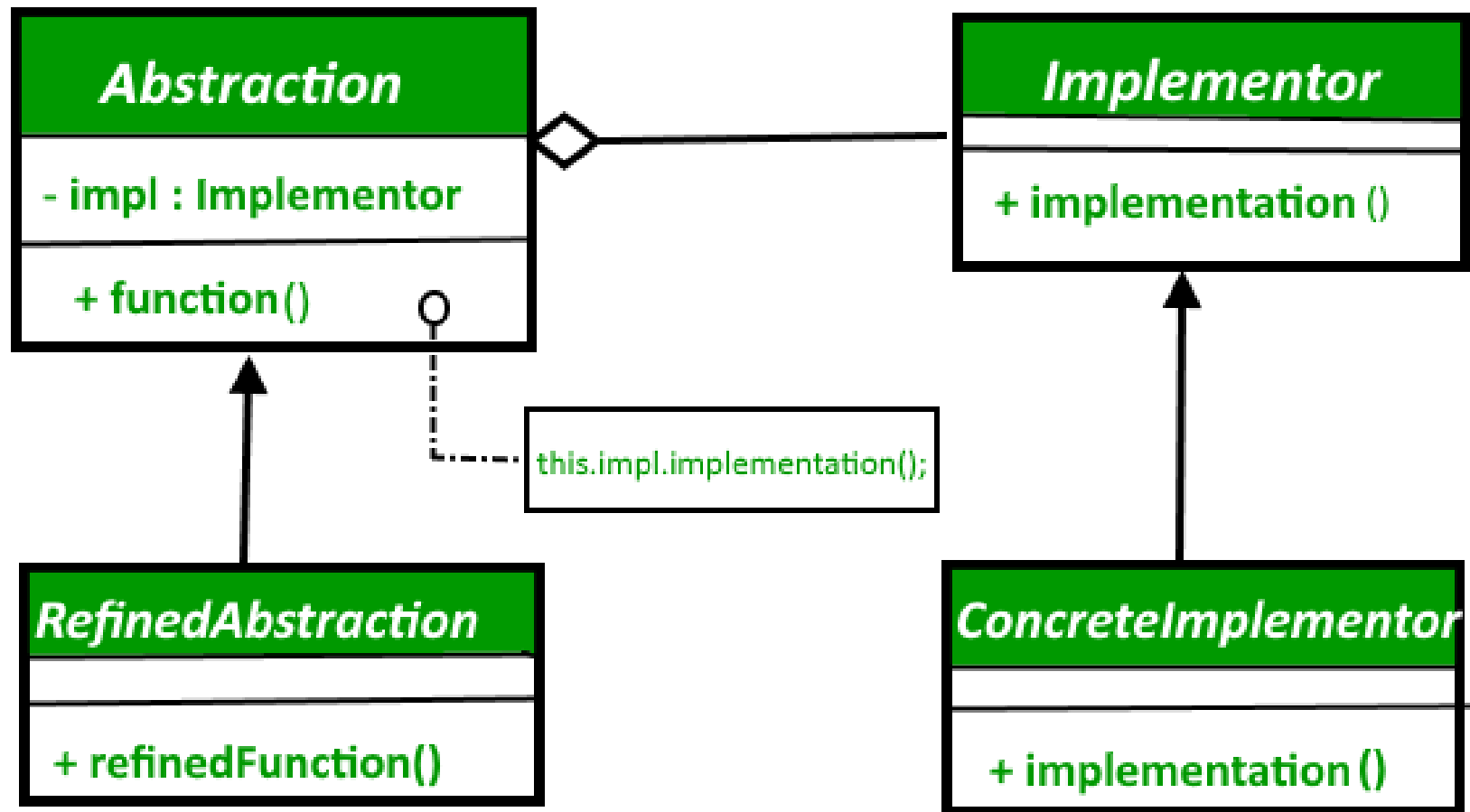
- For solving the problem we defined above, We can use Adapter Method that helps by creating an Adapter object. To use an adapter in our code,
- Client should make a request to the adapter by calling a method on it using the target interface.
- Using the Adaptee interface, the Adapter should translate that request on the adaptee.
- Result of the call is received the client and he/she is unaware of the presence of the Adapter's presence.



2. Bridge Method

- The Bridge pattern is similar to the Adapter pattern except in the intent that you developed it.
- The Bridge is **an approach to refactor already existing code, whereas the Adapter creates an interface on top of existing code** through existing available means without refactoring any existing code or interfaces.
- The motivation for converting your code to the Bridge pattern is that it may be tightly coupled. There is logic and abstraction close together that is limiting your choices in how you can extend your solution in the way that you need.
- E.g., you may have one Car class, that produces a very nice car. But you would like the option of varying the design a little, or outsourcing responsibility of creating the different components.
- The **Bridge design pattern** is used to decouple an abstraction from its implementation, allowing the two to vary independently. This is particularly useful when you need to **create different variations of a class in a flexible, scalable way without causing an explosion of subclasses.**

- The bridge Method is always considered as one of the best methods to organize the class hierarchy which splits the abstraction from the implementation.

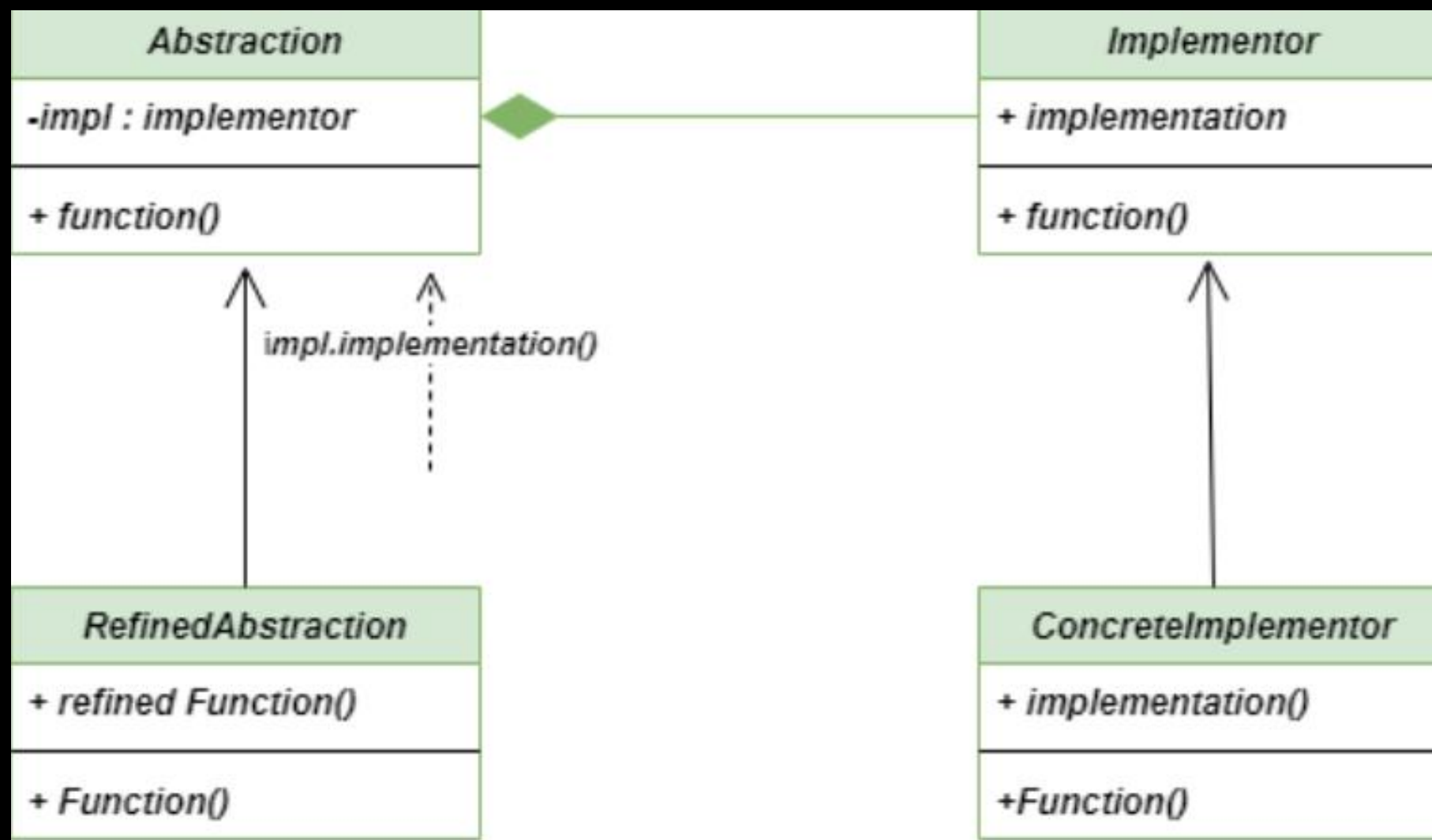


There are 2 parts in Bridge design pattern

- Abstraction
- Implementation

Elements of Bridge Design Pattern

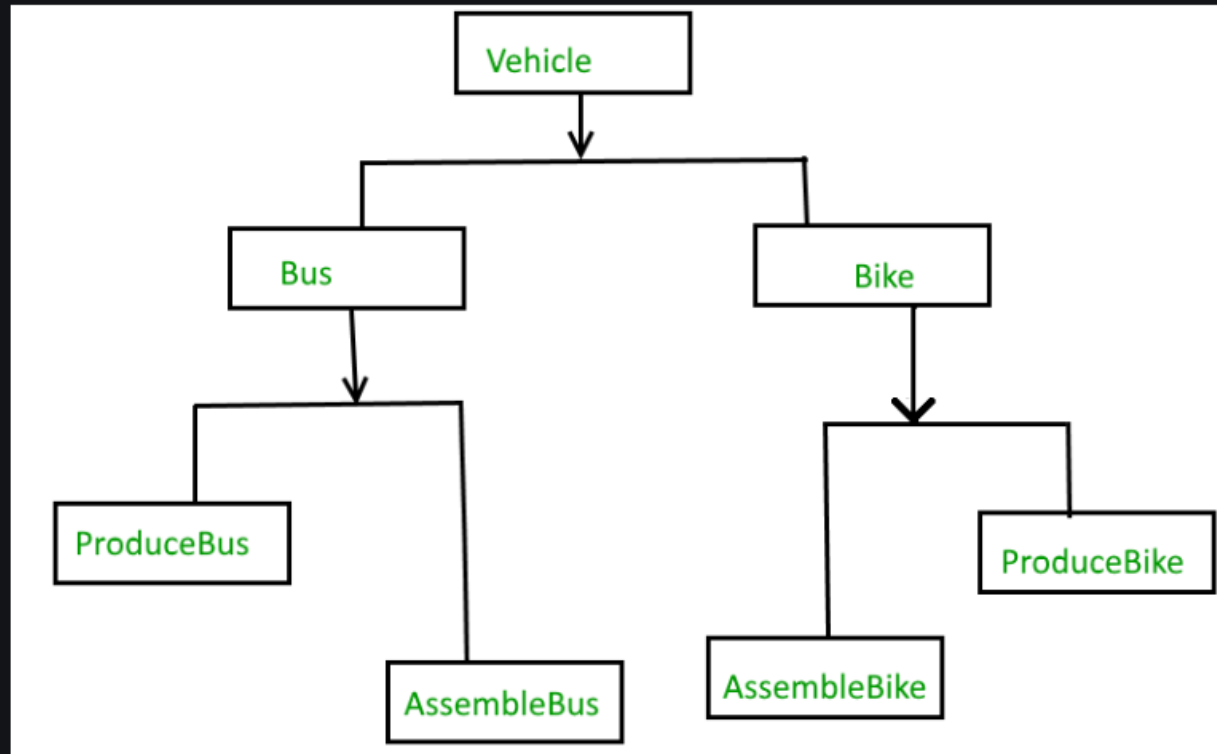
- **Abstraction:** It is the core of the Bridge Design Pattern and it provides the reference to the implementer.
- **Refined Abstraction:** It extends the abstraction to a new level where it takes the finer details one level above and hides the finer element from the implementers.
- **Implementer:** It defines the interface for implementation classes. This interface does not need to correspond directly to the abstraction interface and can be very different.
- **Concrete Implementation:** Through the concrete implementation, it implements the above implementer.



- The Bridge pattern is an application of the old advice, “prefer composition over inheritance”. It becomes handy when you must subclass different times in ways that are orthogonal with one another.

Example:

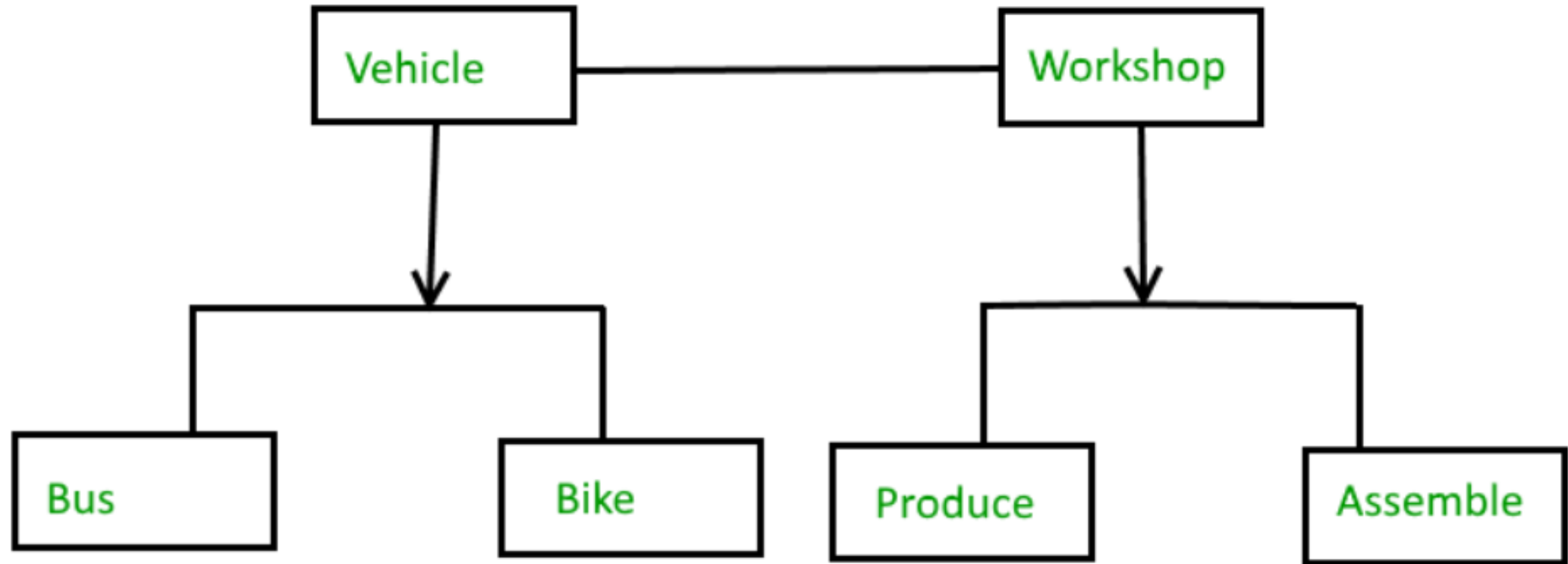
Without Bridge Design Pattern



- But the given solution has a problem. If you want to change the Bus class, then you may end up changing ProduceBus and AssembleBus as well and if the change is workshop specific then you may need to change the Bike class as well.

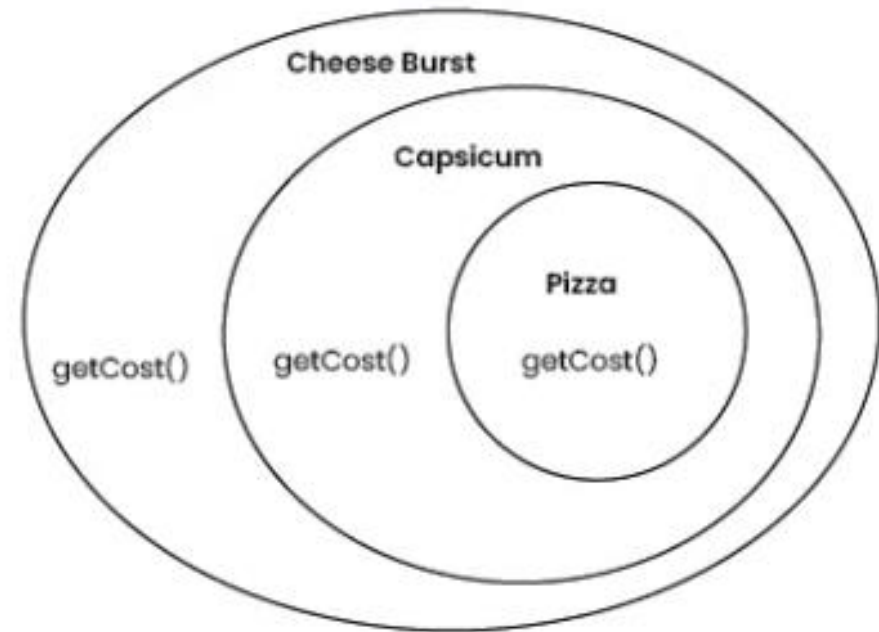
With Bridge Design Pattern

You can solve the above problem by decoupling the Vehicle and Workshop interfaces in the below manner.



Decorator

Design Pattern



Pizza decorated with Capsicum and Cheesburst and total cost of the decorated pizza is calculated.

3. Decorator Method Design Pattern

- Decorator Method is a Structural Design Pattern which allows you to dynamically attach new behaviors to objects without changing their implementation by placing these objects inside the wrapper objects that contains the behaviors.
- It is used to extend the functionalities of classes in a flexible and reusable way.
- It is much easier to implement Decorator Method in Python because of its built-in feature. It is not equivalent to the Inheritance because the new feature is added only to that particular object, not to the entire subclass.
- **Example:** Consider a video streaming platform where users can watch movies and TV shows. Each video content may have additional features or options available, such as subtitles, language preferences, video quality options, and audio enhancements.

Characteristics of the Decorator Pattern

- This pattern promotes flexibility and extensibility in software systems by allowing developers to compose objects with different combinations of functionalities at runtime.
- It follows the open/closed principle, as new decorators can be added without modifying existing code, making it a powerful tool for building modular and customizable software components.
- The Decorator Pattern is commonly used in scenarios where a variety of optional features or behaviors need to be added to objects in a flexible and reusable manner, such as in text formatting, graphical user interfaces, or customization of products like coffee or ice cream.

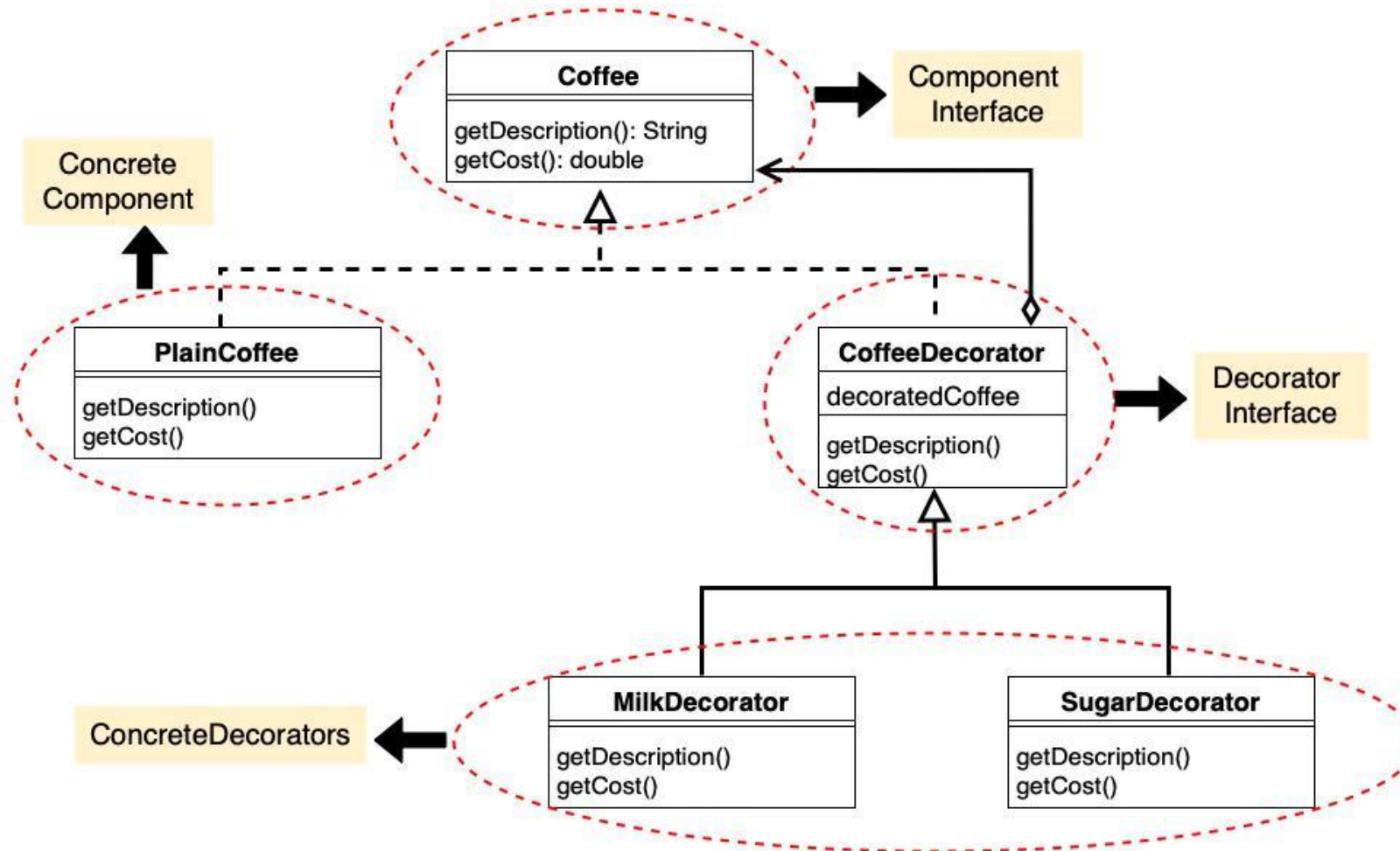
Key Components of the Decorator Design Pattern

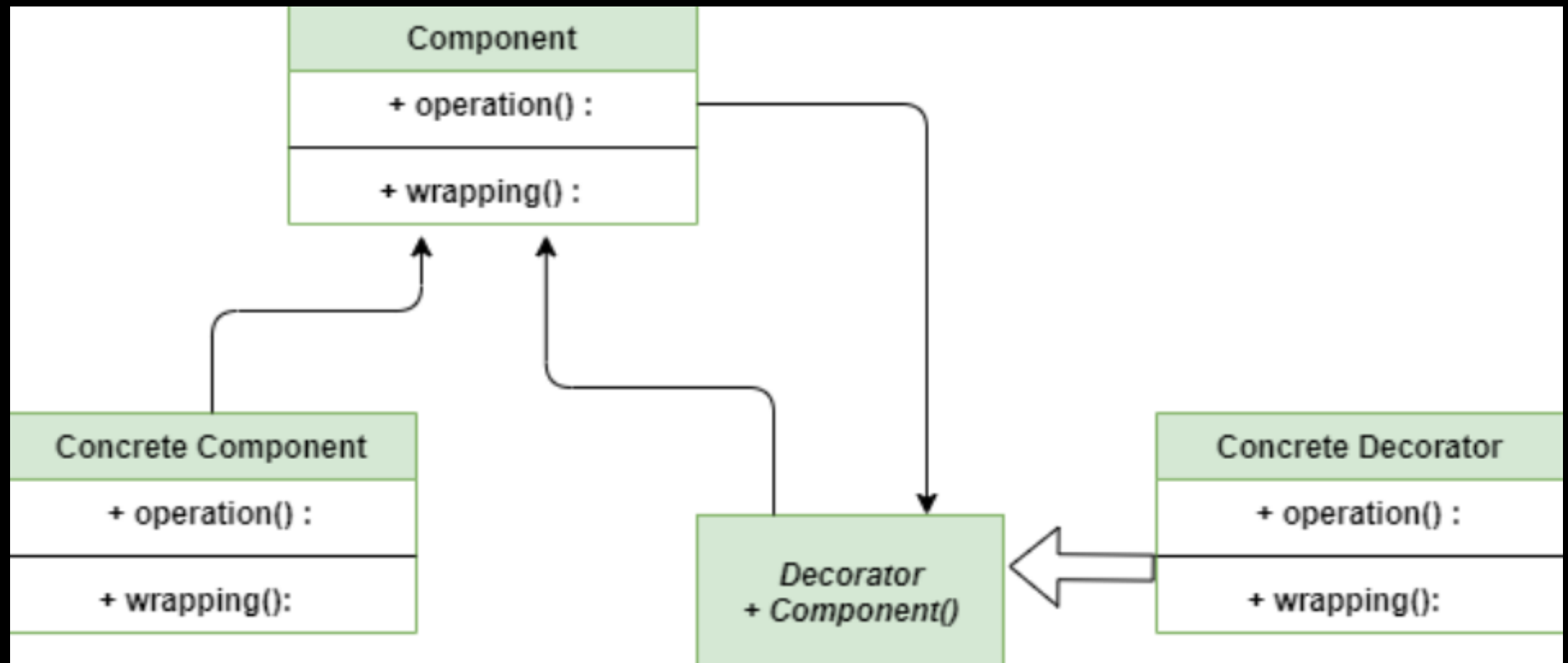
- **Component Interface:** This is an abstract class or interface that defines the common interface for both the concrete components and decorators. It specifies the operations that can be performed on the objects.
- **Concrete Component:** These are the basic objects or classes that implement the Component interface. They are the objects to which we want to add new behavior or responsibilities.
- **Decorator:** This is an abstract class that also implements the Component interface and has a reference to a Component object. Decorators are responsible for adding new behaviors to the wrapped Component object.
- **Concrete Decorator:** These are the concrete classes that extend the Decorator class. They add specific behaviors or responsibilities to the Component. Each Concrete Decorator can add one or more behaviors to the Component.

Example:

- Suppose we are building a coffee shop application where customers can order different types of coffee. Each coffee can have various optional add-ons such as milk, sugar, whipped cream, etc. We want to implement a system where we can dynamically add these add-ons to a coffee order without modifying the coffee classes themselves.
- Using the Decorator Pattern allows us to add optional features (add-ons) to coffee orders dynamically without altering the core coffee classes. This promotes code flexibility, scalability, and maintainability as new add-ons can be easily introduced and combined with different types of coffee orders.

Class Diagram of Decorator Design Pattern





Python Decorator

- Decorators in Python are a powerful and flexible way to modify or extend the behavior of functions or methods. A decorator is a function that takes another function as an argument and returns a new function that adds some kind of functionality.
- They allow you to "wrap" a function or method, adding additional functionality before and/or after the wrapped function runs, without permanently modifying the function itself.

```
1  def my_decorator(func):
2      def wrapper():
3          print("Something is happening before the function is called.")
4          func()
5          print("Something is happening after the function is called.")
6      return wrapper
7
8  @my_decorator
9  def say_hello():
10     print("Hello!")
11
12 say_hello()
13
```

```
13
14 def log(func):
15     def wrapper(*args, **kwargs):
16         print(f"Calling {func.__name__} with {args} and {kwargs}")
17         result = func(*args, **kwargs)
18         print(f"{func.__name__} returned {result}")
19         return result
20     return wrapper
21
22 @log
23 def add(a, b):
24     return a + b
25
26 add(5, 3)
27
```

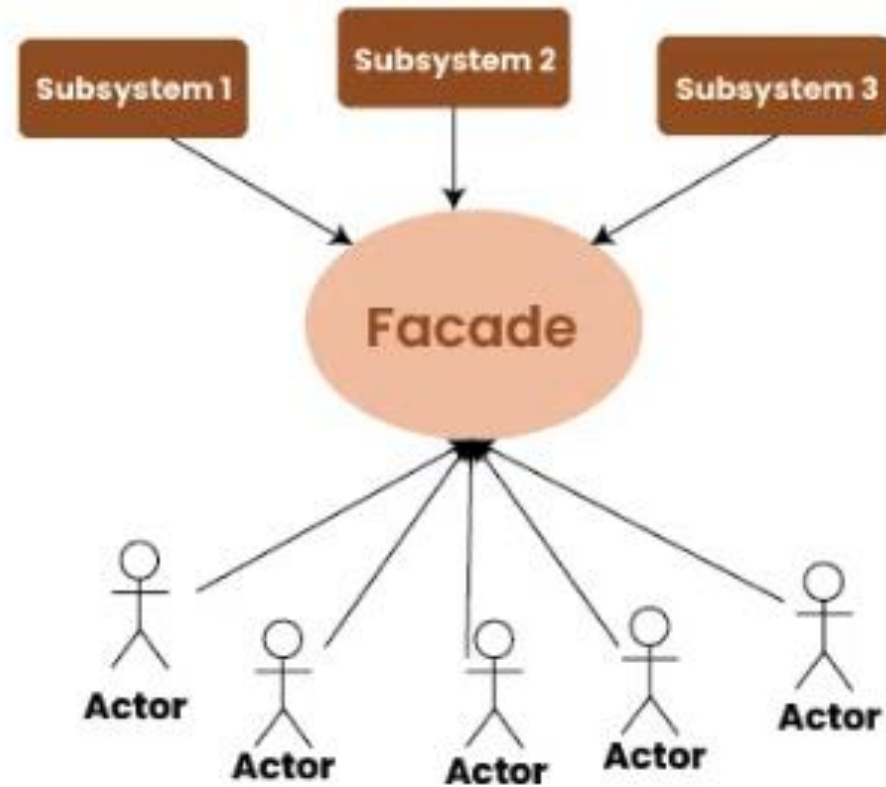
```
29 def require_authentication(func):
30     def wrapper(user, *args, **kwargs):
31         if not user.is_authenticated:
32             raise Exception("Authentication required")
33         return func(user, *args, **kwargs)
34     return wrapper
35
36 class User:
37     def __init__(self, name, authenticated):
38         self.name = name
39         self.is_authenticated = authenticated
40
41 @require_authentication
42 def view_dashboard(user):
43     print(f"Welcome {user.name} to your dashboard")
44
45 user = User("Alice", True)
46 view_dashboard(user)
47
```



```
50  from functools import wraps
51
52  def my_decorator(func):
53      @wraps(func)
54      def wrapper(*args, **kwargs):
55          print("Something is happening before the function is called.")
56          result = func(*args, **kwargs)
57          print("Something is happening after the function is called.")
58          return result
59      return wrapper
60
61  @my_decorator
62  def say_hello():
63      print("Hello!")
64
65  print(say_hello.__name__)
66
67
68
```

Facade Method Design Pattern

in Python



Facade Design Pattern

- The Facade Pattern calls for a facade of your implementation. People don't need to know everything about the underlying implementation. You can create a big class that will fully manage your complex subsystem and just provide the functionalities your user is likely to need.
- The Facade design pattern provides a simplified interface to a complex subsystem. It hides the complexities of the subsystem by exposing a single point of entry, making it easier to use. This is especially useful when dealing with systems with multiple classes that must be used together to perform a task.

Key Components of the Facade Design Pattern

Facade:

- A single class that provides a simplified interface to a complex subsystem. The facade delegates client requests to the appropriate subsystem objects without exposing the underlying complexity.

Subsystem Classes:

- The classes that perform the work of the system. These are the actual components that provide the functionalities. These classes implement the subsystem functionalities. They are not directly accessible by the clients but are used internally by the facade.

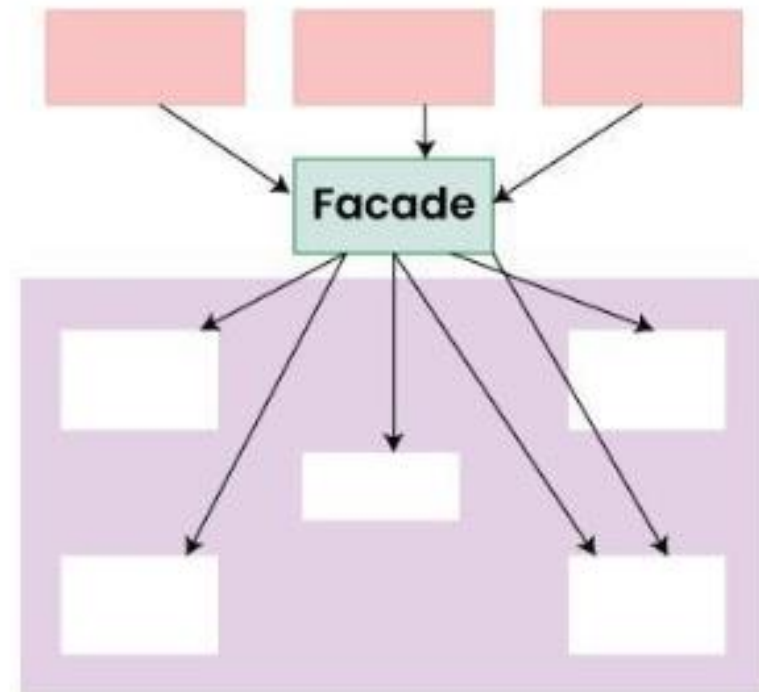
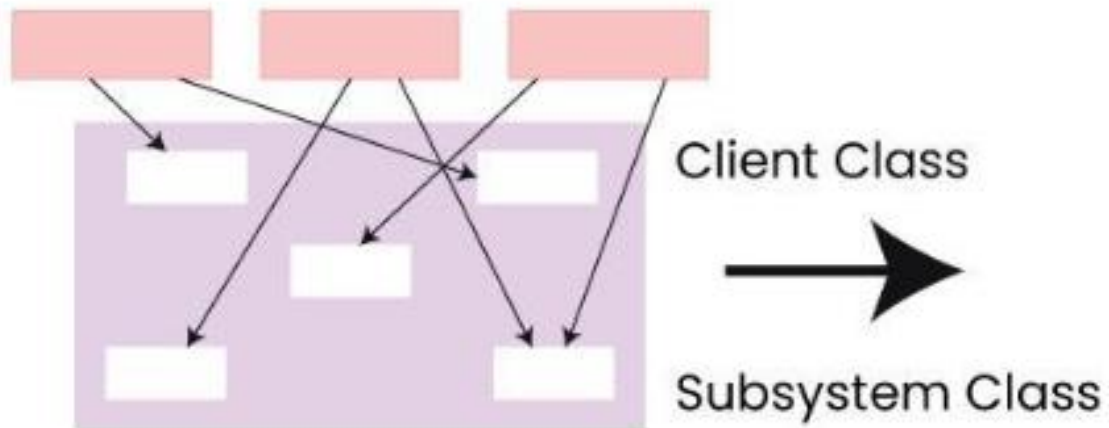
Client:

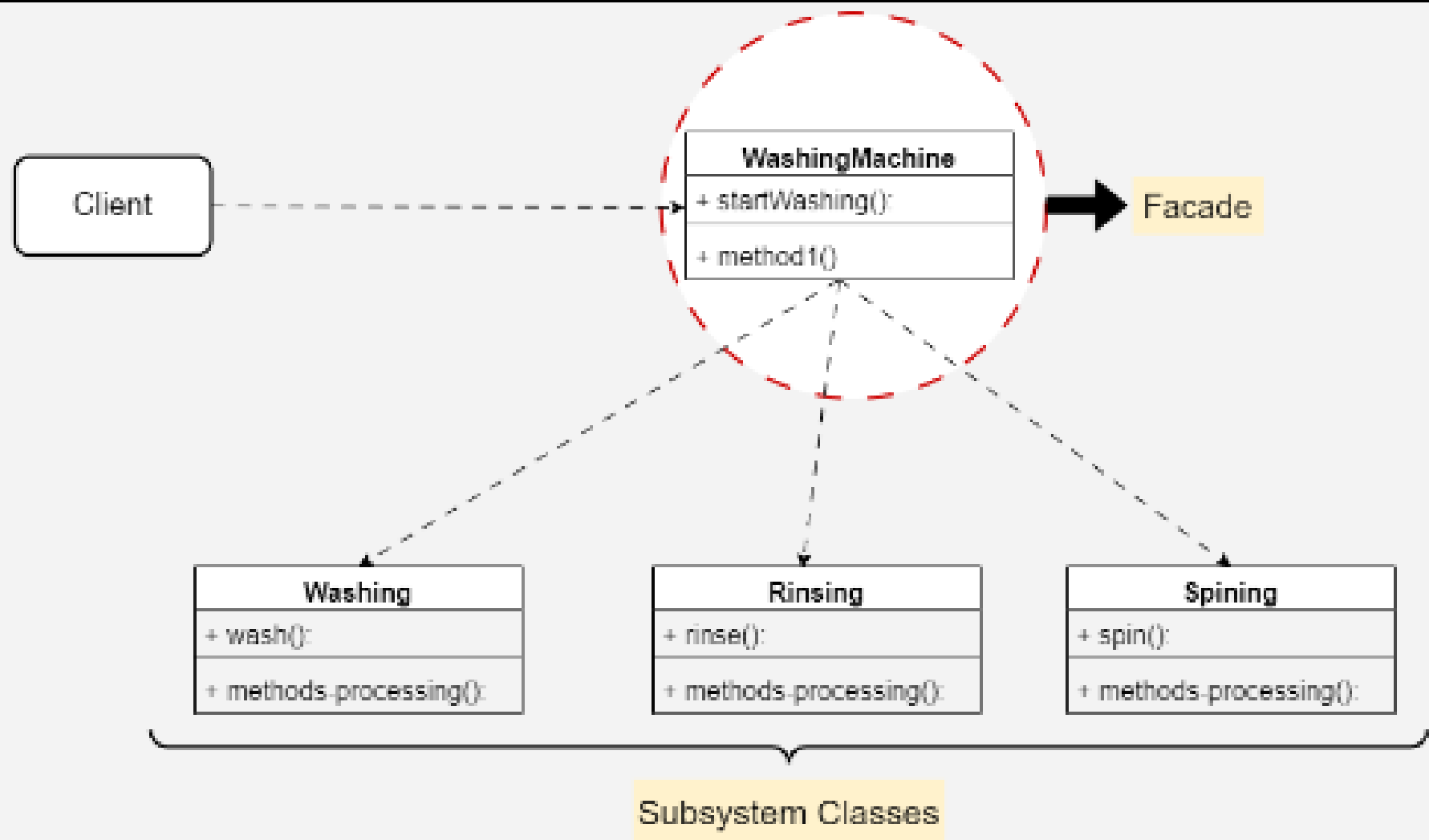
- The external entity that interacts with the subsystem through the facade. The client makes requests to the facade, which in turn calls the appropriate subsystem methods.

Facade Design Pattern

- Facade Method Design Pattern provides a unified interface to a set of interfaces in a subsystem. Facade defines a high-level interface that makes the subsystem easier to use.
-

What is Facade Method Design Pattern







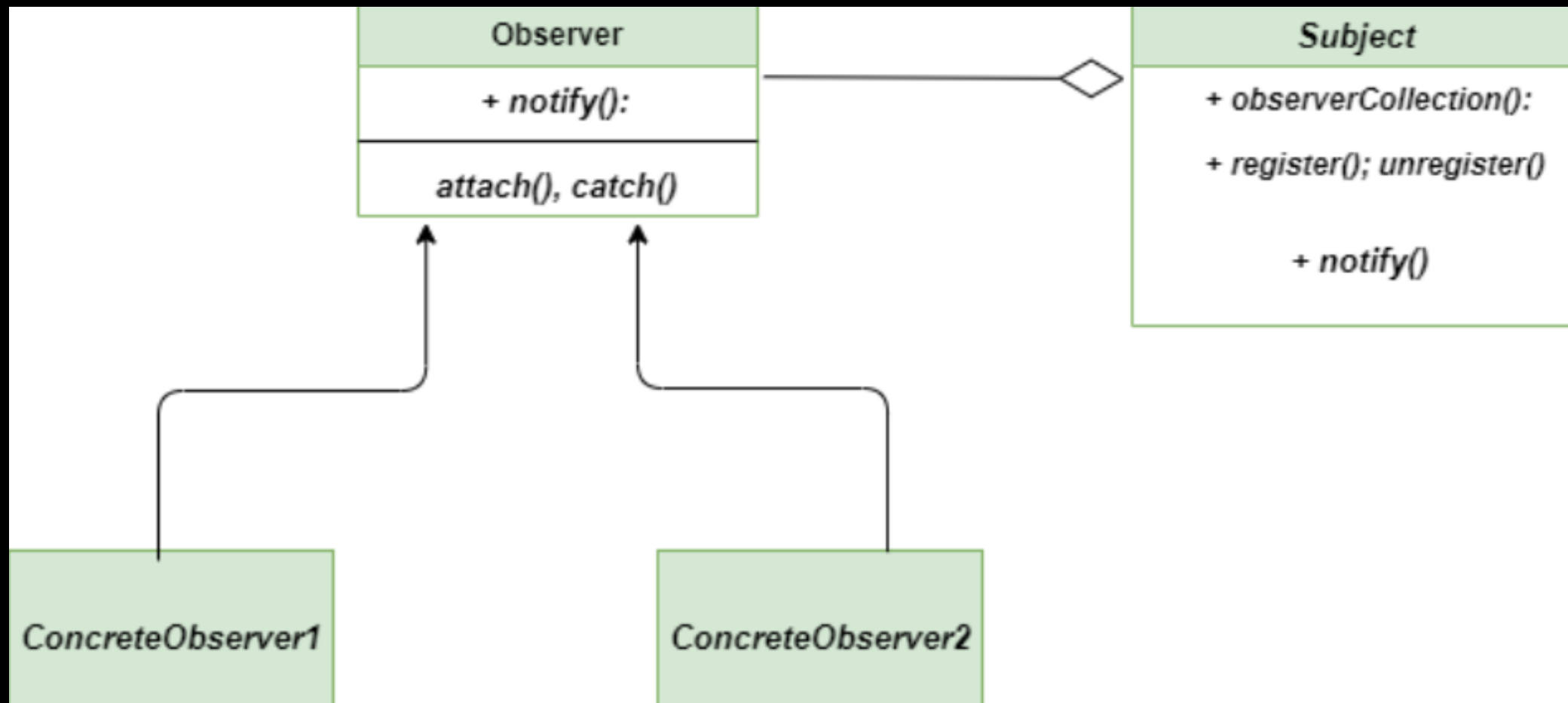
03. Behavioral groups

Behavioral patterns are all about identifying the common communication patterns between objects and realize these patterns. These patterns are concerned with algorithms and the assignment of responsibilities between objects. Chain of Responsibility Method.

1. Command Method
 2. Iterator Method
 3. Mediator Method
 4. Memento Method
 5. **Observer Method**
 6. **State Method**
 7. **Strategy Method**
 8. **Template Method**
 9. Visitor Method
-

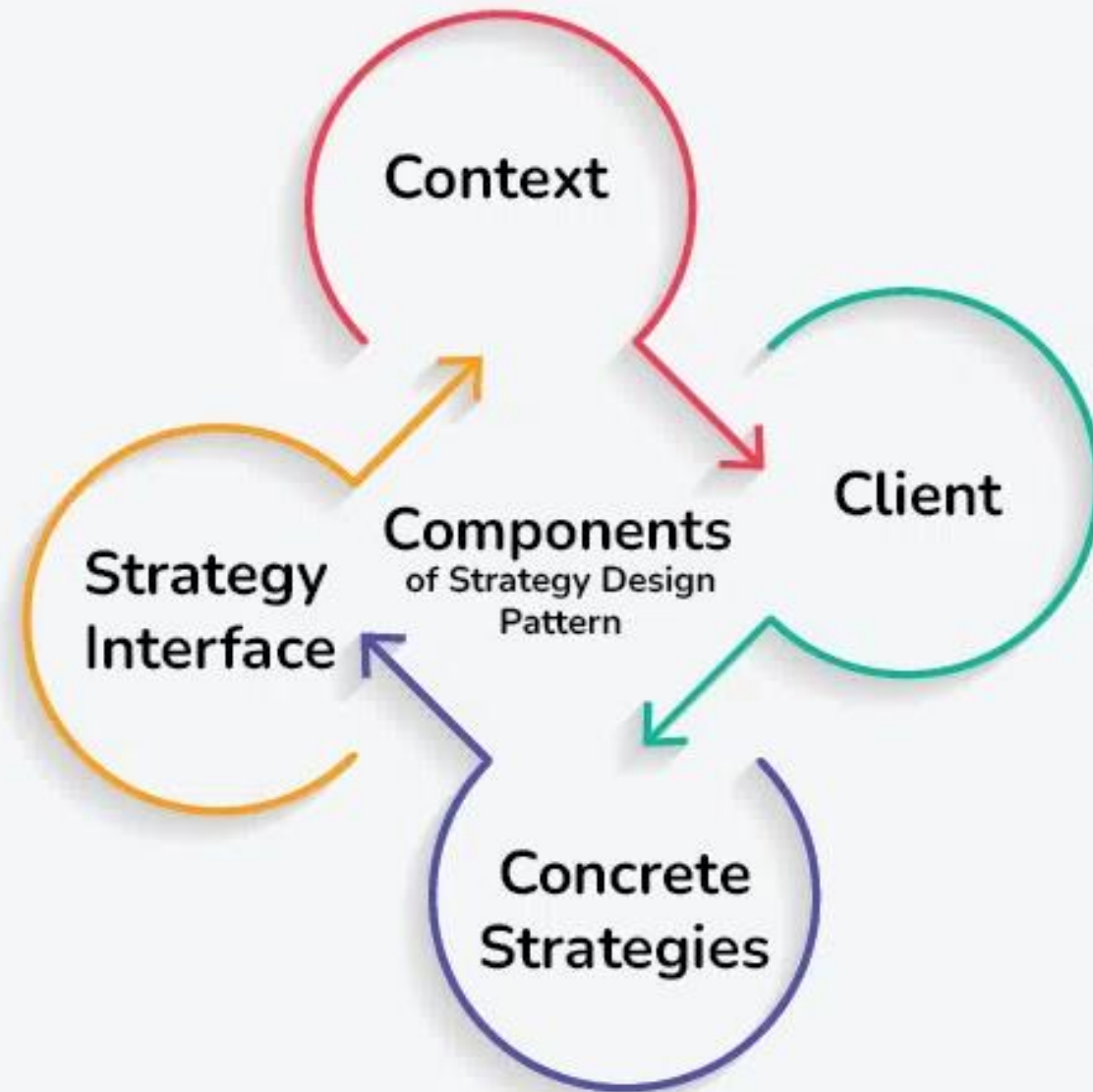
1. Observer Method

- The observer method is a Behavioral design Pattern which allows you to define or create a subscription mechanism to send the notification to the multiple objects about any new event that happens to the object that they are observing.
- The subject is basically observed by multiple objects. The subject needs to be monitored and whenever there is a change in the subject, the observers are being notified about the change.
- This pattern defines one to Many dependencies between objects so that one object changes state, all of its dependents are notified and updated automatically.



2. Strategy Method

- A strategy pattern is a behavioral design pattern that allows the behavior of an object to be selected at runtime.
- In simpler terms, The Strategy Pattern allows you to define a family of algorithms, encapsulate each one of them, and make them interchangeable. This pattern lets the algorithm vary independently from clients that use it.



Components of the Strategy Design Pattern

1. Context

- The Context is a class or object that holds a reference to a strategy object and delegates the task to it.
- It acts as the interface between the client and the strategy, providing a unified way to execute the task without knowing the details of how it's done.
- The Context maintains a reference to a strategy object and calls its methods to perform the task, allowing for interchangeable strategies to be used.

2. Strategy Interface

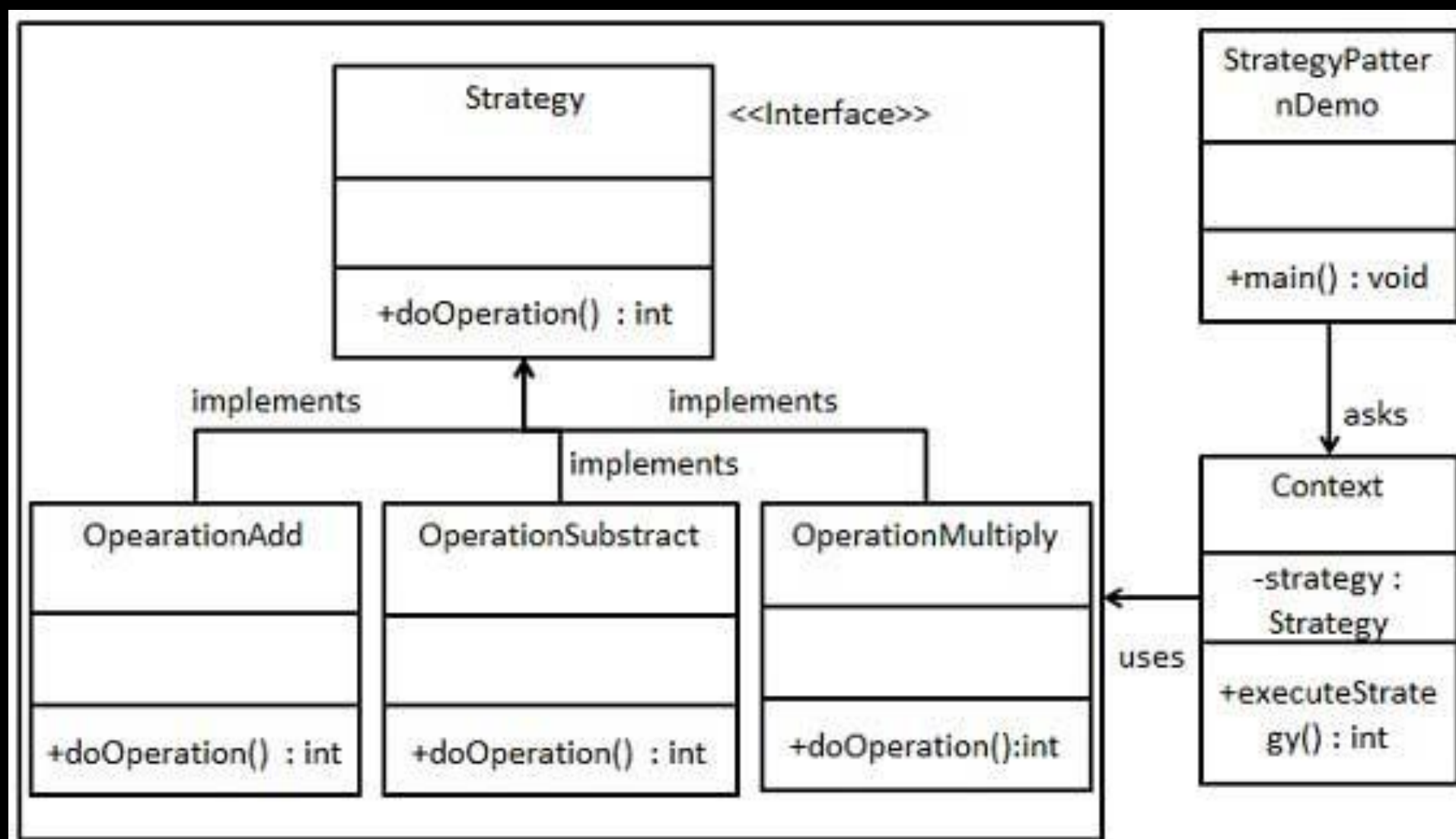
- The Strategy Interface is an interface or abstract class that defines a set of methods that all concrete strategies must implement.
- It serves as a contract, ensuring that all strategies adhere to the same set of rules and can be used interchangeably by the Context.
- By defining a common interface, the Strategy Interface allows for decoupling between the Context and the concrete strategies, promoting flexibility and modularity in the design.

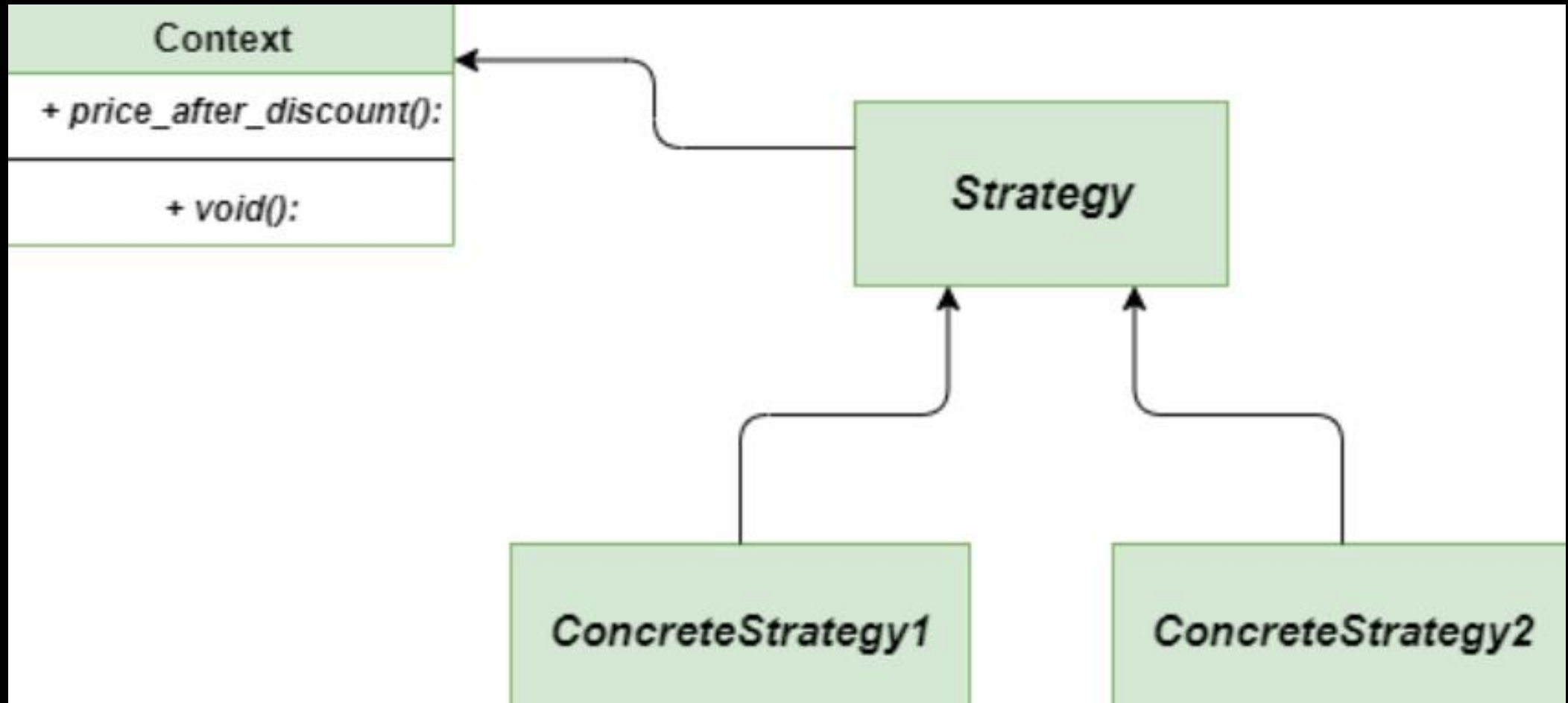
3. Concrete Strategies

- Concrete Strategies are the various implementations of the Strategy Interface. Each concrete strategy provides a specific algorithm or behavior for performing the task defined by the Strategy Interface.
- Concrete strategies encapsulate the details of their respective algorithms and provide a method for executing the task.
- They are interchangeable and can be selected and configured by the client based on the requirements of the task.

4. Client

- The Client is responsible for selecting and configuring the appropriate strategy and providing it to the Context.
- It knows the requirements of the task and decides which strategy to use based on those requirements.
- The client creates an instance of the desired concrete strategy and passes it to the Context, enabling the Context to use the selected strategy to perform the task.



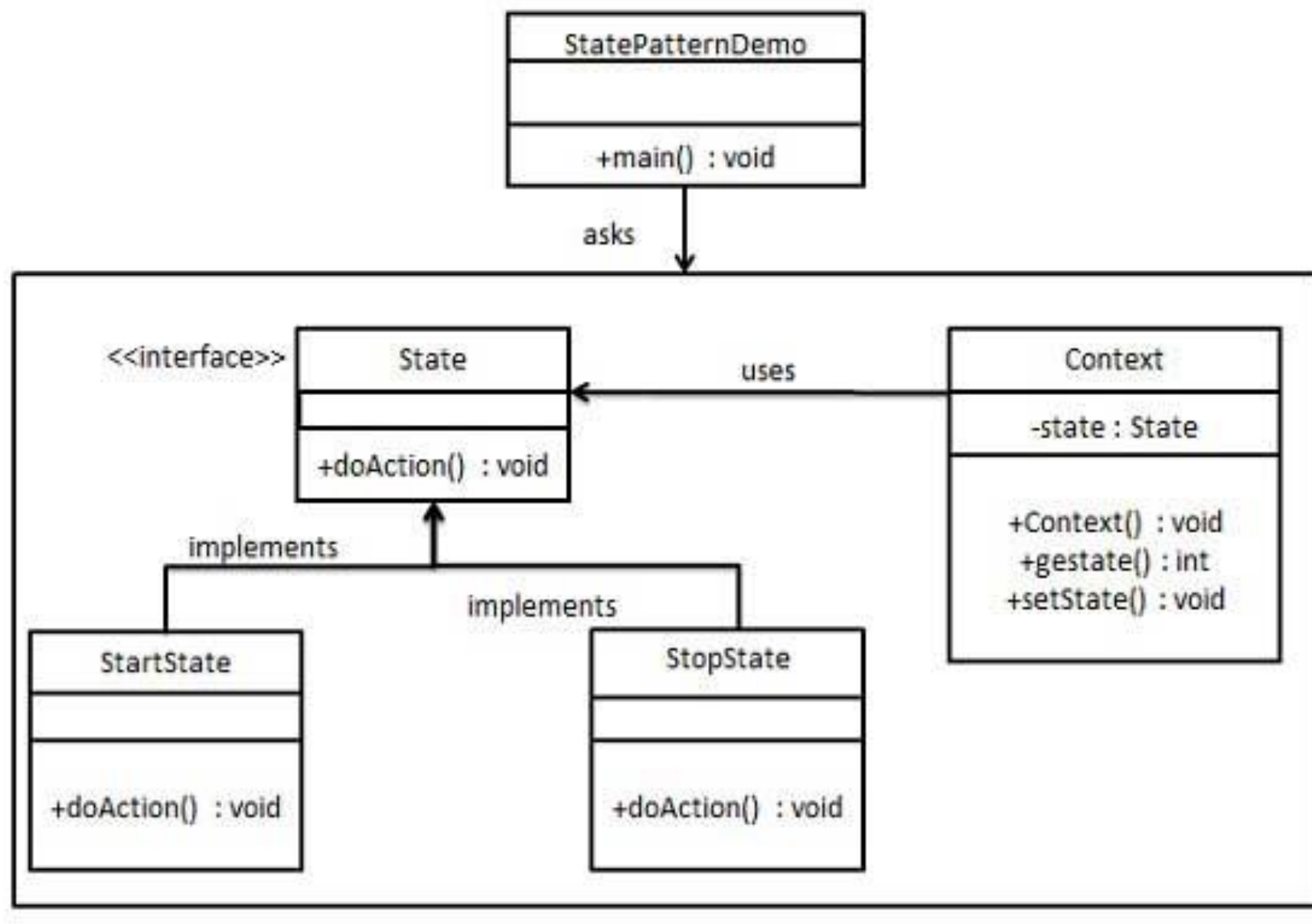


3. State Method

- The State Design Pattern is a behavioral design pattern that allows an object to change its behavior when its internal state changes.
- This pattern is especially useful when an object's behavior needs to be altered based on its state.

Key Components:

- Context: The object that changes its behavior depending on its internal state.
- State Interface: Defines common behaviors that all states should implement.
- Concrete States: Classes that implement specific behaviors of each possible state and may switch the context to other states.



Mediator Method

- Mediator Method is a Behavioral Design Pattern, it promotes loose coupling between objects by centralizing their communication through a mediator object. Instead of objects directly communicating with each other, they communicate through the mediator, which encapsulates the interaction and coordination logic.

Other Behavioral Patterns

Command Method

- Command Method is a Behavioral Design Pattern, it promotes loose coupling between the sender (client) and the receiver (the object that performs the operation) and provides a way to support undoable operations.

Memento Method

- Memento Method is a Behavioral Design Pattern, it provide to save and restore the previous state of an object without revealing the details of its implementation.

Template Method

- Template Method is a Behavioral Design Pattern, it defines the skeleton of an algorithm in a method but lets subclasses alter some steps of that algorithm without changing its structure.

Visitor Method

- Visitor Method is a Behavioral Design Pattern, it is used when you have a set of structured, hierarchical objects and you want to perform various operations on these objects without modifying their classes.

Interpreter Design Pattern

- Interpreter pattern is used to defines a grammatical representation for a language and provides an interpreter to deal with this grammar.